Instinto

Lince's Technical Documentation

Lince Version:

Typst Version: 0.14.2

Documentation Start: 2025-12-15

Documentation Print: 2026-04-04

Days of Construction: 110

Source: Lince

Contents

1. Birth 5
   1.1. Summary 6
   1.2. Installation 7
   1.3. Disclaimers 9
   1.4. Organization 10
   1.5. Tech Stack 10
2. Primeiros Passos 12
3. O que é Lince? 14
   3.1. Lince ajuda você a atender qualquer Necessidade 15
   3.2. O que é uma Necessidade? 15
   3.3. O que é uma Necessidade? 15
   3.4. Como atendemos essas Necessidades com Lince? 15
   3.5. Como atendemos essas Necessidades com Lince? 15
   3.6. Como atendemos qualquer Necessidade? 16
   3.7. Dois lados de atender uma Necessidade 16
   3.8. Antes do atendimento da Necessidade 16
   3.9. Durante o atendimento de uma Necessidade 16
   3.10. Depois do atendimento de uma Necessidade 16
   3.11. Ambos se parecem 16
   3.12. São os dois lados da mesma moeda 16
   3.13. Aplicação centrada em registros 16
   3.14. Como um registro funciona? 17
   3.15. Exemplo de um registro 17
4. Karma 18
   4.1. Se, e, então... 19
   4.2. Se, e, então... 19
   4.3. Se, e, então... 19
   4.4. "Se" é a Condição 19
   4.5. "Se" é a Condição 19
   4.6. "Se" é a Condição 19
   4.7. "Se" é a Condição 19
   4.8. "Se" é a Condição 20
   4.9. "Se" é a Condição 20
   4.10. "Se" é a Condição 21
   4.11. Dois cenários 21
   4.12. Cenário 0 21
   4.13. Cenário 2 21
   4.14. Con-se-quên-cias! 21
   4.15. Encadeando 22
   4.16. Cascateando de novo 22
   4.17. Exemplo prático 22
   4.18. Fluxo 22
5. Transações 23

5.1. Dinheiro ... 24
5.2. Sonhador ... 24
5.3. Transação entre três partes ... 24
6. Lince ... 25
6.1. Você ... 26
6.2. Você e outras pessoas ... 26
6.3. Órgão ... 26
6.4. Órgãos ... 26
6.5. Lince ... 26
7. Aging ... 27
7.1. Record ... 28
7.2. Collection ... 29
7.3. Views ... 30
7.4. Karma ... 31
7.5. Examples ... 34
7.6. Configuration ... 37
7.7. Transfer Proposals ... 38
7.8. History ... 39
7.9. Dna ... 40
8. Sickness ... 41
8.1. Good practices ... 42
8.2. Bucket ... 43
8.3. Crates ... 45
8.4. Web Components ... 50
9. Sandbox Website Mode ... 59
9.1. What Is Agreed ... 59
9.2. Binary And Feature Direction ... 59
9.3. Primary Model ... 60
9.4. Persistence Model ... 60
9.5. Required New Artifact ... 60
9.6. Required Route Policy ... 61
9.7. Why The Normal Board Client Cannot Be Reused Unchanged ... 62
9.8. Why Widget-Level Discipline Is Not Enough ... 62
9.9. SSE Policy ... 62
9.10. Bridge Direction ... 63
9.11. Edit, Build, Publish ... 63
9.12. Blog And Website Content ... 64
9.13. Criticisms And Rejected Shortcuts ... 64
9.14. Criticisms Of The Remaining Ambition ... 65
9.15. Minimal Sound Slice ... 65
9.16. Kanban ... 65
9.17. Extensions ... 106
10. The Death of Lince ... 127
10.1. Serious Mode ... 128

1. Birth

It's a Lince!

— Docta

1.1. Summary

Lince is a tool for registry, interconnection and automation of Needs and Contributions with open scope. The phrase above is the densest way to explain Lince. The current page serves as an overview, and the subsequent chapters cover it in much more detail. Let's start with the registry part:

Registry
Users will start their Lince application for the first time, and that will create a personal database (for UNIX systems [Linux and macOS] it is located at ' /.config/lince/lince.db').

That database is a Sqlite file, that is reffered by the application as a DNA. That file can be altered to make the app behave differently. The behavior can be of: setting personal/professional tasks, modeling personal items, scheduling computer actions, act out economic trades between parties, and many more, the sky, your imagination and computing power is the limit.

Any app behavior mentioned above can be boiled down to a Need, or a Contribution to a Need. There are many ways to use the application for that end, one of them is the following: the differentiator of need or contribution is a Quantity. If such quantity is negative, that will be a Need. Having −2 Apples means you need to go and get two apples to be neutral, to satiate your needs. The same applies to a habit, having a −1 Quantity in Exercise means you need to exercise. The previous description is the basis of the whole app, everything revolves around it.

Automation
If there is predictability in your Needs, and a fixed frequency is enough to describe them, then a simple automation can be constructed. One can set the automatic every day change of the 'Studying' habit's Quantity to −1. And with another feature, if that Quantity is −1 a 'Command' is sent to the computer that closes all programs that are not related to it.

Another point of automation is the forementioned DNA. One aspect of constructing systems is that many great ideas often are lost with time, memory and the passing away of those contributors. Common patterns and knowledge about production and maintenance of commodities, economic trades, access to resources and ways of living is scattered.

These patterns could be saved into some form of database, that when fed into a program like Lince, will create Needs and Contributions, so users can have a headstart in any endeavor. From production according to a bill-of-materials to morning routines and parties organization, these activities create Needs for gathering resources and acting out tasks.

Contributions to Lince DNAs are like creating blueprints for others to get up and running. They might not be a perfect fit, but allow for great customization. Currently the way of centralizing DNAs is the Github Repository.

Interconnection
With great customizability and automation of people's DNAs, the next barrier is the connection of those individual apps, turning each machine into a p2p node, also called a

Lince Cell. The same way you create triggers and rules for the quantites of your DNA, you can do so for others.

If one Market has 200 apples and you need 5, you can create a Transfer Proposal. The Quantity of money diminishes on your side, and the Apple one increases. On the Market side, a manual price can be inputed and accepted when proposals come, or it can be adaptive. Donations of clothes, and borrowing tools for quick use would be similar exchanges, just without the money part.

1.2. Installation

You can run Lince as an executable, here are the releases. Pick the latest one for your machine and operating system. After the download, unzip it and run the binary:

```
./lince
```

For a fresh Ubuntu VPS, the smallest practical always-on setup is the public container image from `ghcr.io/lince-social/lince`:

```
sudo apt-get update
sudo apt-get install -y docker.io
sudo systemctl enable --now docker

sudo docker run -d \
  --name lince \
  --restart unless-stopped \
  -p 6174:6174 \
  -v lince-data:/var/lib/lince \
  ghcr.io/lince-social/lince:rolling
```

This keeps Lince running in the background, stores its state in the `lince-data` volume, and starts it again after reboot. `podman run` can use the same image and flags if you prefer Podman instead of Docker.

If your VPS uses `ufw`, open the HTTP port:

```
sudo ufw allow 6174/tcp
```

After that, open `http://YOUR_SERVER_IP:6174`.

If you prefer not to use containers, the shortest binary path on a fresh Ubuntu VPS is:

```
curl -fsSL https://lince.social/install.sh | bash
~/.local/bin/lince --listen-addr 0.0.0.0:6174
```

The explicit `--listen-addr 0.0.0.0:6174` matters on a VPS because the HTTP frontend defaults to `127.0.0.1:6174`.

Or you can install from the crates.io repository with:

```
cargo install --locked lince
```

This also publishes internal `lince-*` helper crates that the binary depends on. They are implementation details for installing the binary, not a separate public API.

Supported feature shapes for crates.io installs are:

- default HTTP + Karma: `cargo install --locked lince`
- HTTP only: `cargo install --locked lince --no-default-features --features "http"`
- Karma only: `cargo install --locked lince --no-default-features --features "karma"`
- TUI only: `cargo install --locked lince --no-default-features --features "tui"`
- TUI + Karma: `cargo install --locked lince --no-default-features --features "karma,tui"`
- GUI only: `cargo install --locked lince --no-default-features --features "gui"`
- GUI + Karma: `cargo install --locked lince --no-default-features --features "karma,gui"`

The frontend features are mutually exclusive:

- `http`
- `tui`
- `gui`

`karma` can be combined with one of them, or used on its own.

If you prefer to compile the source code, the easiest way is to download the repo and run:

```
cargo run
```

If you want local authentication enabled during setup, create or edit `~/.config/lince/lince.toml` and set:

```
[auth]
enabled = true
```

When auth is enabled, Lince will require an admin user to exist. If none exists yet, run `lince` once in an interactive terminal so it can prompt you to create the initial admin account.

If you don't have Cargo installed, you can install it with Rustup. It comes with Rust and its toolchain. If `cargo` isn't in your `PATH`, add it to your shell configuration (`~/.bashrc` or `~/.zshrc`) so you don't have to type the full path every time (i.e., `~/.cargo/bin/{program}`). Run this in your terminal:

```
echo 'export PATH="$HOME/.cargo/bin:$PATH"' >> ~/.bashrc
echo 'export PATH="$HOME/.cargo/bin:$PATH"' >> ~/.zshrc
```

This uses Typst to generate the documentation. One way of running the project is with Mise, the command runner that installs the programs you need to run. To get Mise, run:

```
curl https://mise.run | sh
```

Then run `mise typst docs preview` for the main documentation preview.

Lince Config Whenever you start Lince, it checks if a `lince.toml` file exists. It is in the config directory of your system, plus the lince/ directory.

Properties that may be in there are:

- `auth.enabled`: turns local authentication on or off.
- `auth.secret`: the secret used to sign the local auth session. Lince can generate it for you on first run.

Example:

```
[auth]
enabled = true
```

Bucket storage is not configured in `lince.toml`. It lives in the `configuration` table inside the app database, in the row with `quantity = 1`.

To configure a bucket, set these fields there:

- `bucket_enabled`: set to `1` to enable bucket storage.
- `bucket_username`: the access key / username for the bucket service.
- `bucket_password`: the secret key / password for the bucket service.
- `bucket_uri`: the bucket endpoint, such as `http://localhost:9000` for a local S3-compatible server.
- `bucket_name`: optional bucket name. If omitted, Lince uses `lince`.
- `bucket_region`: optional region. If omitted, Lince uses `us-east-1`.

If you are editing the database manually, update the active configuration row so it looks like this:

```sql
UPDATE configuration
SET bucket_enabled = 1,
    bucket_username = 'ACCESS_KEY',
    bucket_password = 'SECRET_KEY',
    bucket_uri = 'http://localhost:9000',
    bucket_name = 'lince',
    bucket_region = 'us-east-1'
WHERE quantity = 1;
```

1.3. Disclaimers

All the repositories in the Lince ecosystem have the GNU GPLv3 license. Lince is a non-profit project and crowdfunding is the source of development compensation:

GitHub Sponsors | Patreon | Apoia.se

Lince tries to facilitate and automate the connection between people and resources, by transforming needs and contributions into data. The gains and losses related to the interaction, such as transportation, production and services themselves, remain the responsibility and risk of the parties involved.

1.4. Organization

The workflow of contributing to the Lince ecosystem can be of simply making a Pull Request. But if one wants to have certainty of developing a wanted feature, the best way is to see the DNA of Lince programming tasks.

The online resource to communicate is the Discord.

The git repositories hosted currently in the GitHub organization mean this:

- lince: the app's code and this documentation.
- lince-social.github.io: the website, at lince.social.
- dna: the databases for different Lince behaviors.
- lingua: A future programming language Lince evaluates to execute karma, transactions and more.
- .github: general stuff, like photos, icons and information about GitHub sponsors.
- den: git submodules for all the repos in one place.
- rustart: A template to start a Rust project.

General Links

1.5. Tech Stack

Backend:
- Programming Language: Rust
- Database: Sqlite | Easiest to replicate the concept of DNA. Way too simple, way too reproduceable.
- Database Driver: SQLx | Used for raw and typesetted queries

Architecture:
- Application
  - Services for anything complex, business ruly, that calls repositories or manipulates data.
- Domain:
  - Entities:
    - Clean: the most accurate representation of the main drivers of the application. The source of truth of Lince's concepts.
    - Dirty: structure of data that comes in or goes out through the endpoints, or an alias for a data structure as a Type so it doesnt clutter the screen.
- Infrastructure:
  - Database:
    - Management: database connection, migrations and schema.
    - Repositories: traits and their implementations for database operations like get, update…
  - Utils: functions for quality of life, like logging.

  - Cross_cutting: dependency injection for a collection of many methods in different layers. Mostly for single database connection initiation.
- Presentation
  - GPUI: GPU accelerated Rust dependency used by Zed.

(Some Rust) Learning Resources

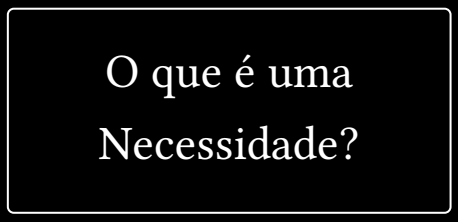
1. Rust Book: chapters 1–6 will give you a simple base if you just want to hack something together. Passing through the entire book, doing the examples and understanding them deeply, will set you up for great vibe coding.
2. Rustlings: for when you are done with the book. You will complete exercises on a broad spectrum of the language's features.
3. Rust for Rustaceans: for a deeper dive in the language, good practices and crate recomendations.
4. This Week in Rust: a weekly newsletter about Rust projects, crates and language changes. Amazing for keeping up to date with the ecosystem, very useful.
5. Youtube Channels: Jon Gjengset | No Boilerplate | The Rustagen | Tsoding | Code to the Moon | fasterthanlime | chris biscardi | Mike Code | Developer Voices
6. Podcast: Rustacean Station
7. Discord, Matrix and many more online communities are very welcoming to newbies.

2. Primeiros Passos

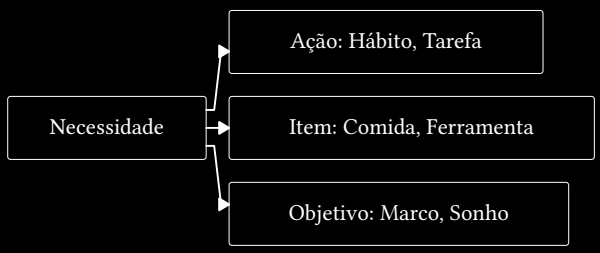
???

3. O que é Lince?

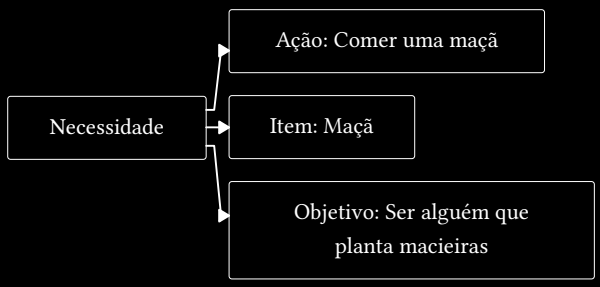
3.1. Lince ajuda você a atender qualquer Necessidade



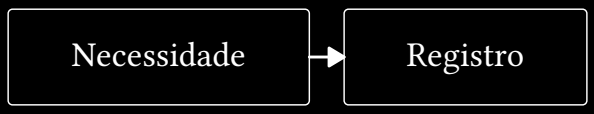
3.2. O que é uma Necessidade?



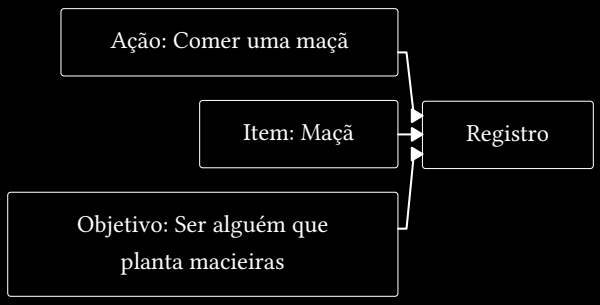
3.3. O que é uma Necessidade?



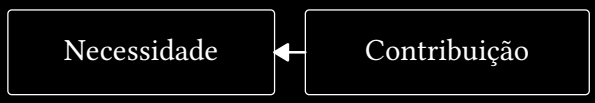
3.4. Como atendemos essas Necessidades com Lince?



3.5. Como atendemos essas Necessidades com Lince?



3.6. Como atendemos qualquer Necessidade?



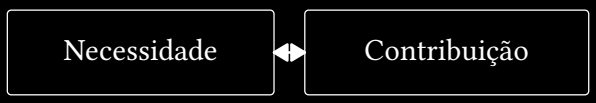
3.7. Dois lados de atender uma Necessidade

3.8. Antes do atendimento da Necessidade

3.9. Durante o atendimento de uma Necessidade

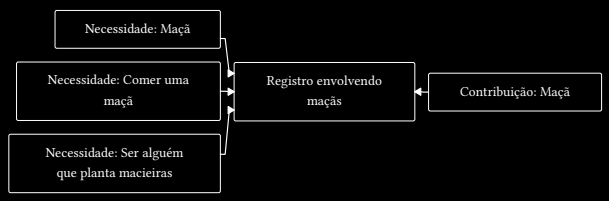
3.10. Depois do atendimento de uma Necessidade

3.11. Ambos se parecem

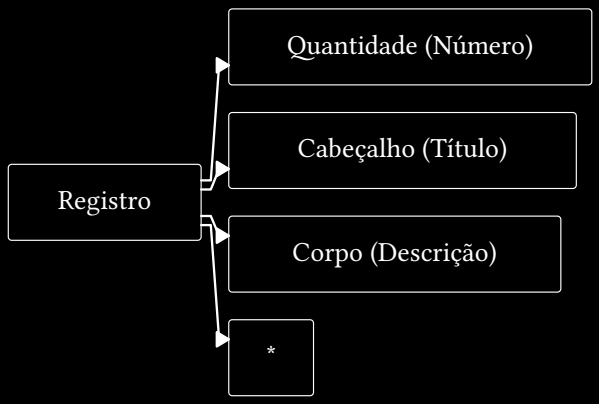


3.12. São os dois lados da mesma moeda

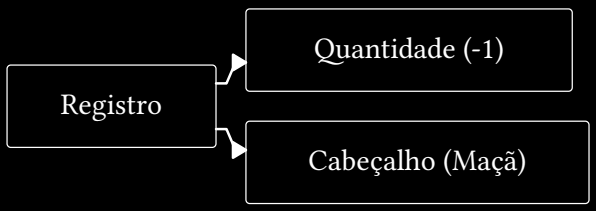
3.13. Aplicação centrada em registros



3.14. Como um registro funciona?



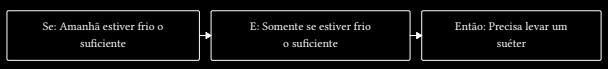
3.15. Exemplo de um registro



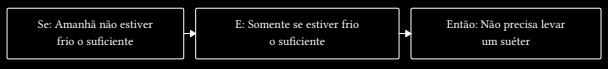
4. Karma

4.1. Se, e, então...

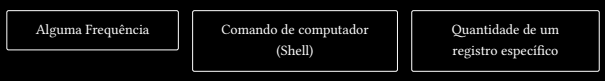
4.2. Se, e, então...



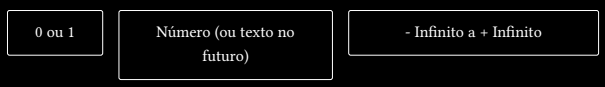
4.3. Se, e, então...



4.4. "Se" é a Condição



4.5. "Se" é a Condição



4.6. "Se" é a Condição

4.7. "Se" é a Condição

4.8. "Se" é a Condição

Condição: Dados > 5

Condição: Quantidade de um registro específico > 5

4.9. "Se" é a Condição

Condição: Dados > 5

Condição: Quantidade de um registro específico > 5

Condição: 2 > 5

4.10. "Se" é a Condição

Condição: Dados > 5

Condição: Quantidade de um registro específico > 5

Condição: 2 > 5

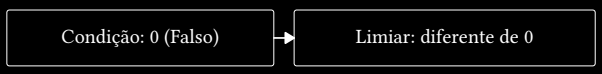
Condição: Falso

4.11. Dois cenários

Condição: 0 (Falso)

Condição: 2 (Quantidade do registro)

4.12. Cenário 0

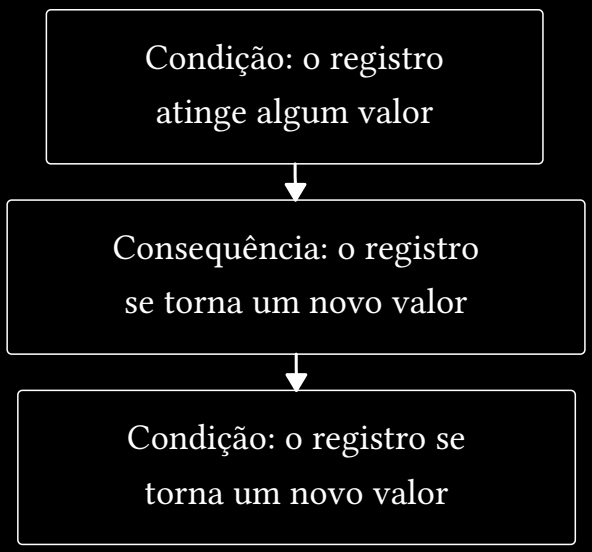


4.13. Cenário 2

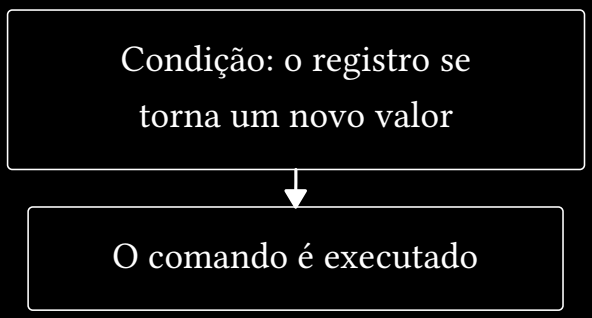
4.14. Con-se-quên-cias!

Registro mudando | Comando Shell | SQL

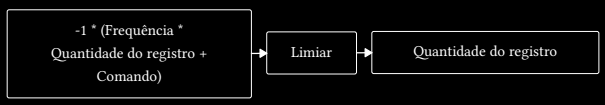
4.15. Encadeando



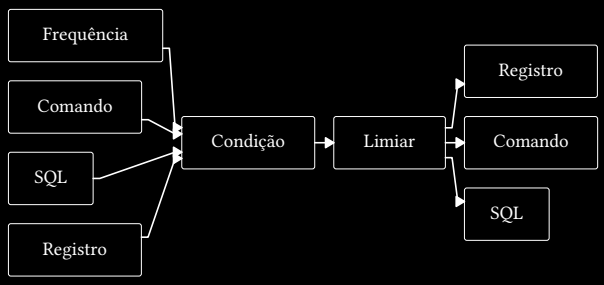
4.16. Cascateando de novo



4.17. Exemplo prático



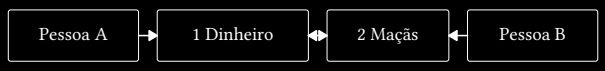
4.18. Fluxo



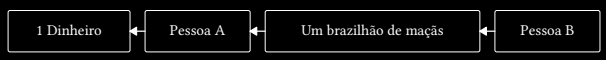
Crédito $\rightarrow$ Débito

5. Transações

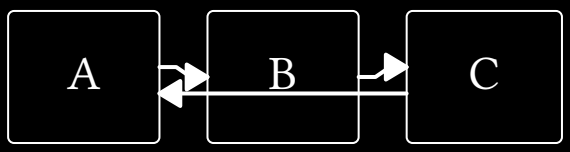
5.1. Dinheiro



5.2. Sonhador



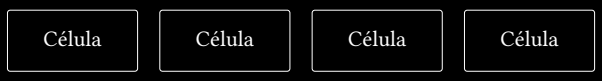
5.3. Transação entre três partes



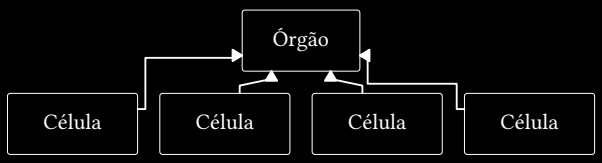
6. Lince

6.1. Você

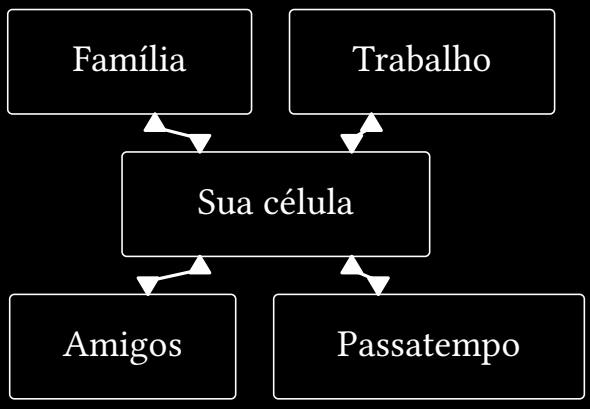
6.2. Você e outras pessoas



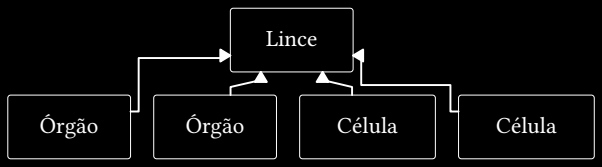
6.3. Órgão



6.4. Órgãos



6.5. Lince



7. Aging

Ouch! My back!

— Sedentary Person, Modern

This chapter explains Lince's components in more detail. It is the worse chapter by far. The project is too much in an Alpha stage and the Schema of the Database changes too much and a definitive one hasn't been chosen (was Postgres at first, now it's SQLite). Some of the properties in here might be out of date or not implemented yet. Explanation of the tables *should* be mostly focused on the ideas, the solutions brought to a problem yet to be found.

7.1. Record

record
id
quantity
head
body
location
time

Lince is centered on the 'record' table, but like, according to the creator… like… what does he know?

A 'record' can be anything: an action, an item, a plan.

record	DATA TYPE
id	INTEGER
quantity	FLOAT
head	TEXT
body	TEXT
location	3D POINT (still thinking)

'id's are automatically generated.

'quantity' represents the availability of the record, if negative it is a Necessity, if positive, a Contribution, zero makes it.

'head' and 'body' are meant to be parts of a whole, where one can be used for a short summary and the other a description, or one has all the information and the other holds tags for filtering through views. With a pubsub protocol, one can send a short information of the record, in this case it can be the head, and put the rest in the body. Only those interested in the head will ask for the body of the record. That way the minimum amount of information is sent over the network, making it faster and stuff, I think.

'location' is an important information for interactions outside of computers (they exist, it's insane) or any other use you want to give it. 'time' is a property like location, not implemented yet. It could be a hard-coded property that maps to a concept in the real world. Not a generic one like quantity. It's probably better to just have multiple custom properties attatched to a record, like cost of time, location, cost of money as data on the

user's DNA, not as code in Lince repo. If we do it in a generic way, each record can have many different properties Lince doesn't have to predict to fit many use cases. The question is, how to we interlink a record with those properties? Other records? Such properties need to have the effect of their names: negative Time in Record A needs to remove that cost of time from the user's Time Record's quantity?

record	DATA TYPE	USER INPUT
id	INTEGER	
quantity	FLOAT	−1
head	TEXT	Eat Apple
body	TEXT	
location	POINT	

So, for an example, imagine that you like apples and you want to create a task to eat it today.

You create a 'record', giving it '−1' to the 'quantity', for that action is a Necessity in your life right now, and 'Eat Apple' to the 'head'.

—

record	DATA TYPE	USER INPUT	ACTUAL RECORD
id	INTEGER		1
quantity	FLOAT	−1	−1
head	TEXT	Eat Apple	Eat Apple
body	TEXT		NULL
location	POINT		NULL

The end result, on the database, is this record.

—

Here is an example of different possible records for individual items and actions.

id	quantity	head	body	location
1	−1	Eat Apple	NULL	NULL
2	−1	Apple	Item, Food	NULL
3	0	Brush Teeth	Action, Hygiene	NULL
4	3	Toothbrush	Item, Hygiene	NULL
5	−1	Meditate	Action	NULL

7.2. Collection

COLUMNS	DATA TYPE

id	INTEGER
name	TEXT
quantity	INTEGER

A Collection is the name of a list of Views. Only one Collection is active at a time, with quantity 1, the rest is 0.

7.3. Views

COLUMNS	DATA TYPE
id	INTEGER
name	TEXT
query	TEXT

Views are ways to select data, to see what records exist with filters or ordering… They can also be used to see what Collections, Karma, Configurations… exist.

A view will have a name, so it is human readable and a query to display different data.

The query is made with SQL allowing you to select what columns you want to see, filtered, ordered and much more, just the way you want it.

The ‘query’ of a view can also be a special code, like ‘karma_view’, in the HTML (Web Browser) frontend. It is a specially constructed html to interact with Karma in a more complete and sane way, to bring ergonomics to day-to-day Lince. Instead of returning an automatically constructed table with data from an SQL query, the Backend adds customized HTML components to the page sent to the user (normal frontend behavior).

Examples:

Let’s grab the record table shown before.

id	quantity	head	body	location
1	−1	Eat Apple	NULL	NULL
2	−1	Apple	Item, Food	NULL
3	0	Brush Teeth	Action, Hygiene	NULL
4	3	Toothbrush	Item, Hygiene	NULL
5	−1	Meditate	Action	NULL

A view of things you need to buy may look like this:

COLUMN	VALUE
id	1
name	Buy

query	SELECT * FROM record WHERE LOWER(body) LIKE ‘%item%’ AND quantity < 0

And in this case, will display only:

id	quantity	head	body	location
2	−1	Apple	Item, Food	NULL

Since no other records with a body containing ‘Item’ (lower, upper, pascal case…) exist with a negative quantity.

Collection View

COLUMNS	DATA TYPE
id	INTEGER
quantity	INTEGER
collection_id	INTEGER
view_id	INTEGER

A Collection View is an intermediate table for grouping Views into a Collection. The quantity is the order in which they appear and if positive they are shown, negative ones are not.

7.4. Karma

karma	DATA TYPE
id	INTEGER
quantity	INTEGER
name	TEXT
condition_id	INTEGER
operator	TEXT
consequence_id	INTEGER

Karma is a condition checker and an action taker. A constructor of if/then, behavior, inside your DNA. It does it by selecting data or running commands, then if the data fits some criteria, changing records, running commands and more. It replaces a syntax that references parts of Lince with it’s actual values, then evaluating it as a mathematical equation and according to the Operator. If the operator is ‘=’ it lets only non zero values take effect, if it is ‘=*’ it doesnt have that constraint.

This is the hardest thing to understand in Lince and also the most useful. The full description, plus examples, will take place a little further ahead. This process is called a Delivery and it is run every 60 seconds, and sometimes instantly when wanted.

Karma Condition

COLUMN NAME	DATA TYPE
id	INTEGER
quantity	INTEGER
name	TEXT
condition	TEXT

A Karma Condition is something checked by replacing parts of the string (text) with real values. A record with id 1 has a quantity of 5. When we set a Karma Condition to be 'rq1' we are saying the value evaluated will be 5 (at that Delivery).

Karma Consequence

COLUMN NAME	DATA TYPE
id	INTEGER
quantity	INTEGER
name	TEXT
consequence	TEXT

The Karma Consequence works the same way as Condition but instead of getting values it is responsible for setting what is supposed to change. It can be the activation of a Shell/SQL command, the changing of the value of a Record and more; in the future it would be cool to make transactions, changing Quantities of other tables like Configuration and Collection Views, maybe even altering other Karma, or changing DNAs.

Frequency

frequency	DATA TYPE
id	INTEGER
quantity	INTEGER
day_week	INTEGER
months	INTEGER
days	INTEGER
seconds	INTEGER
next_date	STRING
finish_date	STRING
catch_up_sum	INTEGER

The frequency table holds account of a fixed period for returning a value of 1 or more. If a frequency occurs every 1 day and 60 seconds it might start, for example at 2030-01-01 10:00:00. When that time comes, if this Frequency exists in a Karma Condition it will be checked and updated, setting next_date to 2030-01-02 10:01:00.

The catch_up_sum is a multiplier, if many days have passed since the last check like 4, instead of returning one each minute, it catches up to the present in 'next_date' and returns the periods passed or only one.

In Karma, Frequencies are represented by the letter ‘f’. So setting in a Condition ‘−1 * f1’ means Karma Delivery will check the returned number for the specific Frequency with Id 1 and multiply that by ‘−1’.

Command

command	DATA TYPE
id	INTEGER
quantity	INTEGER
name	TEXT
command	TEXT

The Command is a Shell command you can run in a bash Shell.

Example:

id	quantity	command
1		touch grass.el

It is referenced in Karma Condition and/or Consequence as the letter ‘c’, followed by the id number, so this example would be ‘c1’. The command above creates the file grass.el.

Query

COLUMN NAME	DATA TYPE
id	INTEGER
quantity	INTEGER
query	TEXT

The Query is an SQL command you can run and affect your current DNA.

Example:

```
["id", "quantity", "query"], ["1", "", "Robert'); DROP TABLE users; −"],
```

It is referenced in Karma Condition and/or Consequence as the characters ‘sql’, followed by the id number, so this example would be ‘sql1’.

Sum

sum	DATA TYPE
id	INTEGER
quantity	INTEGER
record_id	INTEGER
interval_relative	BOOL
interval_length	INTERVAL
sum_mode	INTEGER

end_lag	INTERVAL
end_date	TIMESTAMP

This table is responsible for returning a sum of change. Change of a record's quantity over time. You have the 'record_id' to know what record to look for. 'interval_length' is a period, can be 1 day, 8 months, etc. If 'interval_relative' is TRUE, will sum all the changes ending now, starting at the past based on the 'interval_length' (ex: 'interval_length' of 8 months and 'interval_relative' is TRUE will make it look at the past 8 months). If FALSE an 'end_date' will need to be supplied. so the past 8 months ending in a certain date. 'end_lag' will give a space between now and the end point of a relative interval. So if the 'interval_length' is 8 months and 'interval_relative' is TRUE, an 'end_lag' of 2 months will make the sum look at the 2 months back to 10 months back, starting 10 months ago, ending 2 months ago. 'sum_mode' tells Lince what to sum, if 0 it's a delta, essentially doing a quantity now - quantity earlier. If it's negative, it sums all negative changes, positive, idem.

7.5. Examples

So you now know the parts of Karma. It's time to put everything together. Lets take this example:

Record

id	quantity	head	body
1	0	Exercise	

Frequency

id	quantity	days	next_date
1		1	2030-01-01 10:00:00

Condition

id	quantity	name	condition
1		Daily	−1 * f1

Consequence

id	quantity	name	consequence
1		Exercise (Automatically set)	rq1

Karma

id	quantity	name	condition_id	operator	consequence_id
1		Daily Exercise (Automatically set)	1	=	1

With this data, every Karma Delivery (60 seconds) will check the validity of each condition. Frequency with id 1 ‘f1’ will return 0 in all minutes of the day safe for one time, where it will return 1. Since the operator of the Karma with id 1 is ‘=’ the expression in it’s Condition ‘−1 * f1’ will almost always be ‘−1 * 0’ and will not have a consequence.

In the Karma delivery that is past the next_date of Frequency, the next_date is updated and the Condition is ‘−1 * 1’. Since that is equal to ‘−1’ the Consequence ‘rq1’ will change the quantity of Record with Id 1, turning it into ‘−1’.

We go from this:

id	quantity	head	body
1	0	Exercise	

To this:

id	quantity	head	body
1	−1	Exercise	

One could make Conditions like ‘(rq1 - 1) * f1’ and a Consequence of ‘rq2’ so the number would not be fixated at −1, but would decrease by one every day based on rq1. The expressions can become very complicated, but unless you are doing finance, joinning multiple sources of data into two or three records like monthly costs/gains and runaway you will do simple expressions.

Example: Command

With the command table

Let’s say you have a command:

id	quantity	name	command
3	1	Radarada	touch grass.el

This command with ID 3 can be referenced in a Karma Consequence with ‘c3’.

Consequence

id	quantity	name	consequence
2		Radarada (Automatically set)	c3

We can reuse the Frequency, and Condition, creating just a Consequence and Karma to automate the running of this command.

Frequency

id	quantity	days	next_date
1		1	2030-01-01 10:00:00

Condition

id	quantity	name	condition
1		Daily	−1 * f1

Karma

id	quantity	name	condition_id	operator	consequence_id
1		Daily Radarada (Automatically set)	1	=	2

This will daily run the Command ‘touch grass.el’.

—

Alternativelly, you can make it so the Command is part of the Condition.

```
echo $(find ~/books/technology -type f | wc -l)
```

This command will output a number, which we can use inside a Condition.

Command

id	quantity	name	command
4		Tech Books Count	echo $(find /books/technology -type f \\| wc -l)
5		Open Current Tech Book	pdfreader /books/techology/current/*

Condition

id	quantity	name	condition
3		Read Tech Task	−1 * f1 * c4

Consequence

id	quantity	name	consequence
3		Open Current Tech Book	c5

Karma

id	quantity	name	condition_id	operator	consequence_id
1		Daily Radarada (Automatically set)	3	=	3

That will automatically open the current technology book one is reading. If the automatic opening is not ideal one can set it so the Consequence is changing the quantity of a Record:

id	quantity	head	body
1	−1	Read Tech Book	

When typing 1 in operation the quantity of this Record will be zero. If you have a Condition that is ‘rq1 == 0’ that will be evaluated to ‘1’ and can trigger the Consequence

‘c5’. Though this method needs one Karma expression to run ‘c5’ if ‘rq1 == 0’ and another to make it so if ‘rq1 == 0’ Condition then ‘rq1’ Consequence, making ‘rq1’ be ‘1’ and not triggering the infinite opening of the Tech Book. In simpler terms: you set two different Karma, one checks if ‘Read Book’ has quantity 0, if yes, make it 1. The other Karma opens the book if the quantity of ‘Read Book’ is zero. So when we set it through zero it will open the book once, for we have a rule that quickly moves it to not zero.

7.6. Configuration

configuration	DATA TYPE
id	INTEGER
quantity	INTEGER
language	TEXT
timezone	TEXT
style	TEXT

The Configuration table alters some behavior of the Lince application. Language and Timezone are what they appear, and style is for setting what colorscheme is used.
Operations

There is a keyboard way of navigating through Lince, changing records and running commands. That is done through the Operations. When one types anything in the frontend, an input will appear.

If you type ‘4c’, a modal for creating a Record will pop up. Typing ‘a’ with a number, like ‘a2’ will make Configuration with id 2 be active, and others inactive, an easy way to change colorscheme. One handy one is ‘s’ with a number, ‘s1’ to run a Shell command.

Just typing a number, like ‘23’ will make the Record with id 23 have it’s quantity become zero.

Writting ‘k3’ will change the active collection to be the one with Id 3, effectivelly changing screens.

Its possible to click in the collection to change it, and setting the quantity to zero by manually editing the field in the table. But those operations are very frequent and can be more ergonomic for those that like to remember shortcuts. Those that dont will have a hard time currently :(

[#] Name	[Key] Action
[0] Configuration	[c] Create
[1] Collection	[q] SQL Query
[2] View	[k] Change Collection
[3] collection_View	[s] Shell Command
[4] Record	[a] Activate Configuration

[5] Karma_Condition
[6] Karma_Consequence
[7] Karma
[8] Command
[9] Frequency
[10] Sum
[11] History
[12] DNA
[13] Transfer

7.7. Transfer Proposals

transfer	DATA TYPE
id	INTEGER
records_received	JSON
records_contributed	JSON
agreement	JSON
agreement_time	TIMESTAMP
transfer_confirmation	JSON
transfer_time	TIMESTAMP

It is possible to find many different ways to divide the world into two. Lince separates it into a Need or a Contribution.

A Transfer Proposal is the process of agreeing to meet a Need or to Contribute. It might be a way of exchanging physical resources, buying services, exchanging information, assigning tasks… It has a few steps: a proposal, an agreement and an action. It is designed to work by Record Quantities between different DNAs.

It can be used as a brainstorm mechanism. If I post a Need (make a Record public as a Need), anyone that wants to Contribute will give a different solution, suggesting to break it down into other Needs and helping with some of them.

Transfer Proposals are a way to generate consensus. If we model Needs and all agree on a Contribution, the next steps for action and automation follow a better agreed upon plan.

‘records_received’ is a collection of records and their quantities that will interact with our records, things you will receive. ‘records_contributed’ are the records you will contribute and their quantities, to the records of other parties, can be more than one party. So you receive 1 in your ‘Apple’ Record’s quantity, and give out 5 moneys to the other party involved.

‘agreement’ is a collection of agreement by all parties involved. ‘agreement_time’ is the moment every party agreed for the conditions of the trade, who will receive what.

‘transfer_confirmation’ is also a collection but with a confirmation from all parties that the transfer was successful, and ‘transfer_time’ for saving the event’s moment.

O melhor fluxo é a modelagem das necessidades, a interação entre pessoas (contribuições) e então a automatização por último.

Como acontece uma proposta de transferencia? Começo com informação. Falo o que poderia contribuir. Me faço ser entendido. Confirmo que essa contribuição atende alguma necessidade. Faço minha parte.

Como deveriam funcionar as contribuições de design e programação pra Lince? Como deveria ser a minha contribuição? Acho que eu começaria com informação. Falo o que poderia contribuir. Me faço ser entendido. Confirmo que essa contribuição atende alguma necessidade. Faço minha parte.

Contribuições para outros Nas contribuições para outros, não contribuo porque tenho autoridade. Não contribuo para uma necessidade inexistente. Não faço para elas o que eu quero e elas não. Explico, proponho, aceitam e faço.

A Lince é um projeto que propoe ser muito, mas ela não se autoexplica e por isso não consegue fazer muito. Eu como contribuidor da Lince a mais tempo preciso fazer mais esforço pra explicar a ideia do que a Lince é hoje. Até ser possível e fácil entender a ideia atual, não posso propor contribuirmos mais para a Lince de alguma forma. Estarei fazendo eco se só eu falar. Quem vai discordar de algo que não entende? Não a gente… As vezes me pergunto… afinal

o que é a Lince?

Quero quebrar minhas necessidades em ações possíveis, quero que outros me ajudem nisso, quero contribuir para elas e quero que me ajudem nisso. Quando o processo estiver decidido e testado quero automatizà-lo.

7.8. History

> This is from the old Postgres Schema and idea of History, it is kept here whilst it is in TODO.

history	DATA TYPE
id	INTEGER
record_id	INTEGER
change_time	STRING
old_quantity	FLOAT
new_quantity	FLOAT

History is a table that automatically logs the change of a record’s quantity, to use the ‘sum’ table. It is possible to do a history of Karma too, that would be useful probably.

7.9. Dna

In practice, each DNA is a different database. In essence, a different Lince behavior. Be it by swapping databases the app is pointing to or aggregating data from multiple databases. It is the data the program has access to understand how to function, what karma rules are applied to what records.

Cough

8. Sickness

'Morbus est medicus naturae': Disease is the doctor of nature, illness forces correction

— ChatGPT (circa 2025)

Lince is not great, it always needs to get better, until it dies, then it doesn't. The way to do it is by giving it medicine (teaching people how to use it) while we cure it (making it simpler and featureful).

For detailed tasks, go to the end of this book.

8.1. Good practices

Making a Lince DNA, modeling items and actions to perform is a continuous effort, if it makes sense to you. With that said, here are some ideas to get started:

– Create a view or more with all the data and configuration for managing your DNA, so Configurations, Collections, Views…

Karma

Until Lince has Deterministic Simulation Testing (DST), which in this context is a way to grab a DNA and simulate years of usage, you have to be mindfull of the Command table you produce, every command you set may possibly break your system if you don't tidy things up. If you have logic of running a command every hour if one record has quantity > X and you forget about it, any simple change will trigger it, so put guardrails for running things, be it Commands, Queries or even changing Record Quantities. Changes might cascade and deliver unforeseen consequences.

With DST this is easier to do, the plan is to have a containerized environment that runs a simulation of your system, isolated from the outside world. Your DNA (your personal configuration of Lince) is a seed that can be run multiple times arriving at the same result (hopefully). Being able to run it with a high speed, changing the date (affecting the Frequency table) and record quantities will bring reproducible results. When you want to add something to your DNA you can check it's effects with a simulation and get info if it breaks anything in an edge case.

Command

Lince works with Sqlite files for it's database. It is recomended to frequently backup your DNAs, weekly, daily or hourly if you are paranoid. If some error or mistake happens, your information is safe.

Personal Tasks – Create Views for understanding your tasks, select the ones that have negative quantity, check this SQL:

```
SELECT * FROM record WHERE quantity < 0 AND LOWER(head) LIKE '%task%'
```

– One can use the 'head' column in 'record' table as a tag holder. So Record's heads with Items and Tasks that have negative quantities appear in my 'Negatives' view.

Finance Currently not very featureful. You can model in Records your monthly earnings and expenditures and make a Karma with a condition of (Records of Earnings) - (Records of Expenditures)

8.2. Bucket

The bucket is one configured object store.

The split between DNA files and general media should be modeled with key prefixes, not separate buckets.

Reserved top-level prefixes in v1:

- `dna/` for published web widget packages derived from records
- `media/` for everything else

This section records the current agreement for moving away from the GitHub `dna` repo as the runtime source of installable widgets.

8.2.1. Agreements

- The long-term source of truth for remote DNA widgets should be bucket objects plus database records, not the GitHub `dna` repo.
- The publication row should be a normal `record`.
- The human-facing listing fields should come from `record.head` and `record.body`.
- The bucket object key should not be stored in `record.body`.
- The bucket object key should live in `record_resource_ref`.
- The client should not scan broad table data from every organ and infer a catalog locally.
- Each organ should expose a dedicated DNA catalog endpoint.
- The host should aggregate those organ-level DNA catalogs for the web UI.
- After approval, installed widgets should land in one shared local directory: `~/.config/lince/web/sand`.
- The older split between `~/.config/lince/web/widgets` and `~/.config/lince/web/sand` is not the desired end state.

8.2.2. Record model

The preferred publication shape is:

- `record` as the main row
- `record_extension` for structured DNA publication metadata
- `record_resource_ref` for the bucket-backed package file

If a category label is reused for discovery, such as `lincedna`, it should be treated only as a coarse label.

It should not be the sole source of truth for whether a record is a published DNA package.

The task-category system discussed in Kanban is still task metadata.

Bucket-backed DNA publication metadata should remain explicit and structured.

Recommended sidecar namespace:

```
record_extension.namespace = "lince.dna"
```

Suggested `record_extension.data_json` shape:

```
{
  "published": true,
  "channel": "official",
  "version": "0.1.0"
}
```

Recommended package file reference:

```
record_resource_ref.provider = "bucket"
record_resource_ref.resource_kind = "sand"
record_resource_ref.resource_path = "dna/official/ka/kanban-record-view/
kanban-record-view.html"
```

Suggested `record_resource_ref.data_json` shape:

```
{
  "slug": "kanban-record-view",
  "filename": "kanban-record-view.html",
  "mime_type": "text/html",
  "sha256": "optional-content-hash"
}
```

The slug belongs with the bucket-backed package reference because it identifies the published artifact, not only the record text.

In v1, a published DNA record should have exactly one primary `record_resource_ref` with:

- `provider = "bucket"`
- `resource_kind = "sand"`
- a `resource_path` under `dna/`

If more resources are later attached to the same record, the `sand` entry remains the canonical install target and the others are auxiliary resources.

8.2.3. Endpoint contracts

Preferred organ-local catalog route:

```
GET /api/dna/catalog
```

Each row should expose the record-facing fields as `head` and `body`, not as `title` and `description`.

Suggested item shape:

```
{
  "record_id": 123,
  "head": "Kanban Record View",
  "body": "Record-centric board with comments, worklog, and resources.",
  "slug": "kanban-record-view",
```

```
  "version": "0.1.0",
  "channel": "official",
  "bucket_key": "dna/official/ka/kanban-record-view/kanban-record-view.html",
  "filename": "kanban-record-view.html",
  "mime_type": "text/html",
  "updated_at": "2026-04-04T12:00:00Z"
}
```

Preferred host aggregation routes:

```
GET /host/packages/dna/catalog
GET /host/packages/dna/{organ_id}/{record_id}/preview
POST /host/packages/dna/{organ_id}/{record_id}/install
```

The preview and install steps should converge into the same package parsing and local materialization flow already used for standalone package imports.

8.2.4. Local install direction

The local machine should keep one shared `sand` directory for web widgets.

That simplifies the user's mental model, but it also means the system must stop treating `sand` as meaning only "built-in official widget".

After this refactor, source metadata should remain explicit even if the filesystem directory is unified.

Useful source labels include:

- built-in official widget
- uploaded local file
- imported from organ DNA catalog

That distinction should live in metadata, not in separate directories.

8.2.5. Migration boundary

The GitHub-backed DNA catalog described elsewhere in the documentation should be treated as transitional.

Once bucket-backed DNA publication and dedicated organ endpoints exist, runtime discovery and installation should use the bucket-plus-record flow described here.

8.3. Crates

This document explains the packaging problem around publishing `lince` to crates.io and what shape the workspace should take if `cargo install lince` is meant to work.

Current code is more authoritative than this document when there is conflict. This note is about distribution shape and release policy, not about changing the internal architecture of the app.

8.3.1. Problem

The repo is a Rust workspace, not a single-package repository. The `lince` binary depends on several internal library crates:

- `application`
- `domain`
- `injection`
- `persistence`
- `utils`
- optionally `web`
- optionally `gui`
- optionally `tui`

That is a normal internal workspace structure, but it matters for crates.io.

If `cargo install lince` is supposed to work from crates.io, Cargo must be able to resolve every dependency of `lince` from crates.io as well. Path-only workspace dependencies are fine for local development, but they are not enough for publishing.

That means there is no middle state where:

- `lince` is published to crates.io
- helper crates remain local-only
- `cargo install lince` still works

If the binary remains split into internal crates, those internal crates must also be publishable.

8.3.2. Why the original generic names are a problem

Originally, the internal packages used very generic package names such as:

- `application`
- `domain`
- `web`
- `utils`

That shape is bad for crates.io for two reasons:

1. These names are global package identities on crates.io, so they are likely to collide with existing or future crates.
2. Even if the names were available, publishing those packages would make them look like general-purpose public libraries, which is not the intent.

The important distinction is:

- package name: the global crates.io identity
- library crate name: the local Rust name used in `use` statements

Those do not need to be the same.

8.3.3. Recommended shape

The clean solution is:

- keep the workspace split as it is
- publish internal packages under `lince-*` names
- keep the local Rust crate names short and unchanged

Examples:

- package `lince-application`, library crate `application`
- package `lince-domain`, library crate `domain`
- package `lince-persistence`, library crate `persistence`
- package `lince-web`, library crate `web`

This preserves the current code shape while making the published identities honest and low-risk.

The intended meaning is:

- `lince` is the user-facing install target
- `lince-*` helper crates exist only because the binary is internally modular
- `lince-*` crates are implementation details, not a reusable public platform

8.3.4. What I would do

I would keep the current workspace architecture and make it explicitly publishable.

That means:

- all internal packages share the same version as `lince`
- workspace dependencies specify both `path` and an exact `=version`
- internal packages use `lince-*` package names
- internal packages keep short `[lib]` names for code imports
- release automation bumps the whole workspace together
- crates publishing happens in dependency order

The publish order should be:

- `lince-utils`
- `lince-domain`
- `lince-persistence`
- `lince-injection`
- `lince-application`
- `lince-web`
- `lince-gui`
- `lince-tui`
- `lince`

Not every crate has to be enabled by default at runtime, but if `lince` depends on it for some publishable feature set, it has to be resolvable by Cargo.

The intended feature matrix for `lince` is:

- default: `karma` + `http`
- alternative frontend: `tui`
- alternative frontend: `gui`
- optional background worker: `karma`

The frontend features are mutually exclusive:

- `http`
- `tui`
- `gui`

So valid user-facing combinations include:

- `http`
- `karma,http`
- `tui`
- `karma,tui`
- `gui`
- `karma,gui`
- `karma`

8.3.5. Why not flatten everything into one crate

There is another possible direction:

- move all internal code into the `lince` package
- stop publishing helper crates entirely

That would also solve the crates.io problem.

I would not do that right now unless there is already a desire to simplify the source tree. It would mix two unrelated changes:

- packaging policy
- codebase architecture

The workspace split already expresses meaningful boundaries inside the project. If those boundaries are still useful for development, testing, and reading the code, then crates.io should adapt to the architecture, not force an unnecessary flattening.

8.3.6. Release policy

There should be two separate release ideas:

- GitHub release: publish binaries and container images
- crates.io release: publish `lince` and the internal `lince-*` helper crates

They are related, but they are not the same action.

The practical release flow should be:

1. bump the workspace version
2. create the Git tag and GitHub release flow
3. publish internal `lince-*` crates in dependency order
4. publish `lince`

This keeps `cargo install lince` aligned with the versioned binary release, while still acknowledging that crates.io propagation is a separate system.

Operationally, that should map to:

- `mise release`
- `mise publish-crates`

8.3.7. What this means for users

Users should think in terms of:

- `cargo install lince`
- downloaded release binaries
- Docker image pulls

Users should not need to know or care that `lince-domain` or `lince-application` exists. Those packages are an implementation artifact of the workspace layout.

8.3.8. What this means for maintainers

Maintainers should treat the internal crates as:

- publishable
- versioned together
- documented as internal
- not promised as stable standalone APIs

That does not require pretending they are a reusable framework. It only requires making the dependency graph publishable.

8.3.9. Default-members note

The workspace has `default-members = ["crates/lince"]`. That means commands run at the workspace root without `--package` may still target `lince` today.

That is convenient, but CI and release automation should still prefer explicit package selection such as:

- `cargo build --package lince`
- `cargo check --package lince`

This avoids accidental behavior changes if the workspace default ever changes later.

8.4. Web Components

This document is the canonical technical specification for web components in Lince.

Current code is more authoritative than this document when there is conflict. This file focuses on ways to get data, ways to use data, and recommended workflows.

8.4.1. Basics

There are two families of web components: standalone imported widgets and official `sand/` widgets. Standalone imported widgets are single HTML documents executed inside an iframe through `srcdoc`. Official `sand/` widgets are Lince-maintained components, usually rendered with Rust + Maud, and should move toward one host-managed runtime contract instead of each widget inventing its own transport.

The main actors are the backend, which owns persisted business data, generic CRUD, saved views, streaming, and semantic actions; the host, which owns widget instance identity, permissions, auth state, board layout, and persisted `widgetState`; the bridge, which connects host and widget; and external systems such as third-party APIs, foreign sites, files, browser APIs, and WASM runtimes.

Treat the bridge as control plane, not bulk data plane. Use it for metadata, state persistence, and runtime flags. Do not use it as the main large-record transport when the backend or official stream should be the source.

Standalone widget package contract

When generating a standalone imported widget, produce one complete self-contained HTML document that is safe to save directly as a `.html` file and execute inside an iframe via `srcdoc`.

Hard requirements:

- do not depend on remote CDNs
- do not depend on remote fonts
- do not assume network access unless the host contract explicitly gives it
- do not render a fake outer browser frame inside the card
- use inline CSS and inline JS
- do not call `parent` window APIs directly
- use the injected bridge if host communication is needed

The embedded manifest block must exist near the top:

```
<script type="application/json" id="lince-manifest">
  {
    "title": "Widget title",
    "author": "Author",
    "version": "0.1.0",
    "description": "Short summary",
    "details": "Longer explanation",
```

```
    "initial_width": 4,
    "initial_height": 3,
    "permissions": []
  }
</script>
```

Manifest fields are straightforward: `title` is the short user-facing name, `author` is the author string, `version` is semantic version text, `description` is the short summary, `details` is the longer explanation, `initial_width` and `initial_height` are integers from `1` to `6`, and `permissions` is the list of host-recognized capabilities.

Host runtime contract

When a widget runs inside the board, it runs in an iframe through `srcdoc`, the host injects a vendored Datastar runtime, the host injects `window.LinceWidgetHost`, and the frame may include dataset metadata.

Do not add a CDN Datastar import.

If internal state must use `localStorage`, namespace it by widget instance:

```
const instanceId = window.frameElement?.dataset?.packageInstanceId ||
"preview";
```

Prefer host-persisted state through the bridge instead of relying only on `localStorage`.

Bridge helper:

`window.LinceWidgetHost`

Bridge surface: `getState()`, `getMeta()`, `getCardState()`, `setCardState(nextState)`, `patchCardState(patch)`, `setStreamsEnabled(enabled)`, `subscribe(handler)`, `requestState()`, and `print(label)`.

Bridge rules are simple: persist long-lived widget ergonomics in `widgetState`, do not store backend credentials there, and do not store backend configuration copies there either.

Datastar bridge example:

```
<div
  data-lince-bridge-root
  data-on:lince-bridge-state="$bridge = evt.detail.bridge; $meta =
evt.detail.meta"
></div>
```

8.4.2. Ways To Get Data

Pick one obvious primary source. Only mix sources when there is a real reason.

Host metadata and bridge state

Use this for widget instance identity, `serverId`, `viewId`, stream enabled state, and persisted `widgetState`. Host metadata may first appear in frame dataset values such as `window.frameElement?.dataset?.linceServerId` and

`window.frameElement?.dataset?.linceViewId`. Once the bridge is ready, treat `meta.serverId`, `meta.viewId`, and `meta.streams.enabled` as authoritative.

Generic table CRUD

Use this when the widget edits ordinary rows in known tables.

Direct backend routes:

```
GET    /api/backend/table/{table}
POST   /api/backend/table/{table}
GET    /api/backend/table/{table}/{id}
PATCH  /api/backend/table/{table}/{id}
DELETE /api/backend/table/{table}/{id}
```

Host-mediated routes for configured remote servers:

```
GET    /host/integrations/servers/{server_id}/table/{table}
POST   /host/integrations/servers/{server_id}/table/{table}
GET    /host/integrations/servers/{server_id}/table/{table}/{id}
PATCH  /host/integrations/servers/{server_id}/table/{table}/{id}
DELETE /host/integrations/servers/{server_id}/table/{table}/{id}
```

Prefer host-mediated routes for remote configured widgets, do not pass `server_id` as a query parameter to `/api/backend/...`, and state clearly which table the widget owns.

View streams

Use this when the widget consumes a saved view as a live dataset.

Current routes:

```
GET /api/backend/sse/view/{view_id}
GET /host/integrations/servers/{server_id}/views/{view_id}/stream
```

Current SSE behavior is simple: events are named, `snapshot` carries the latest JSON payload, and `error` carries a JSON error payload. Reconnect cautiously, keep the UI usable before the first snapshot, do not assume workspace switching remounts the iframe, validate the payload shape, and show compact mismatch UI when the shape is wrong.

Instance-aware official runtime

This is the preferred direction for official `sand/` widgets:

```
GET  /host/widgets/{instance_id}/stream
GET  /host/widgets/{instance_id}/contract
POST /host/widgets/{instance_id}/actions/{action}
```

Use this when the host should own:

- resolving widget config by instance
- consuming backend streams internally
- returning normalized widget-facing contracts
- exposing semantic actions instead of raw multi-table choreography

Short `/contract` example:

```
{
  "widget_kind": "kanban",
  "runtime_shape": "server-shaped board with rich local state",
  "required_columns": {
    "id": "integer",
    "quantity": "integer",
    "head": "text",
    "body": "text"
  },
  "permissions": {
    "read_view_stream": true,
    "write_records": true,
    "write_table": true
  }
}
```

Use `/contract` to describe widget-facing data, not raw storage internals.

Provisioned backing resources

Some widgets cannot start with a pre-existing `viewId`. They need the host to create or bind resources first.

Preferred provisioning direction:

```
GET  /host/widgets/{instance_id}/contract
POST /host/widgets/{instance_id}/provision
POST /host/widgets/{instance_id}/actions/{action}
```

Provisioning should answer which server the widget is bound to, whether provisioning is allowed, whether resources already exist, which `record_id`, `record_extension_id`, and `view_id` belong to the widget, and whether the stream is ready.

Chess is the simplest example: one `record` for game identity, one `record_extension` row for game state JSON, and one `view` that streams the record plus sidecar payload.

Example widget-owned JSON:

```
{
  "history": [],
  "current": {}
}
```

The host should own provisioning and linkage. The widget should own the meaning of the JSON.

Local-only or foreign data

Use this when the widget does not depend on Lince domain data as its main source. Typical examples are clocks, local notes, weather, Spotify control, imported HTML wrappers, and terminal or WASM surfaces. Keep the runtime honest, do not pretend a foreign integration is a normal Record widget, and use `widgetState` for local ergonomics when that state should survive reloads.

8.4.3. Ways To Use Data

Use the simplest storage model that still keeps the feature queryable and maintainable.

Core table rows

Current stable core truth:

- `record`: `id`, `quantity`, `head`, `body`
- `view`: `id`, `name`, `query`

Keep `record` generic. Do not keep adding task-specific columns to `record` unless they become central to Lince itself.

Sparse sidecar JSON

Use `record_extension` for optional, versioned, widget-specific or feature-specific metadata.

Recommended table:

```sql
CREATE TABLE record_extension (
    id INTEGER PRIMARY KEY,
    record_id INTEGER NOT NULL REFERENCES record(id) ON DELETE CASCADE,
    namespace TEXT NOT NULL,
    version INTEGER NOT NULL DEFAULT 1,
    data_json TEXT NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(record_id, namespace),
    CHECK (json_valid(data_json))
);
```

Good uses:

- `task.categories`
- `task.schedule`
- `task.effort`
- `task.type`

Do not store widget UI state there.

Relations and repeated data

If the feature points to its own entity table, use relations instead of scalar metadata.

Recommended relation table:

```sql
CREATE TABLE record_link (
    id INTEGER PRIMARY KEY,
    record_id INTEGER NOT NULL REFERENCES record(id) ON DELETE CASCADE,
    link_type TEXT NOT NULL,
    target_table TEXT NOT NULL,
    target_id INTEGER NOT NULL,
    position REAL,
    data_json TEXT,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(record_id, link_type, target_table, target_id),
    CHECK (data_json IS NULL OR json_valid(data_json))
);
```

Example:

- use `record_link` to `app_user` for assignment when users live in their own table
- only fall back to scalar assignment metadata when no identity table exists

Dedicated feature tables

Use dedicated tables for repeated or lifecycle-heavy data.

Examples:

- comments: use `record_comment`
- worklogs: use `record_worklog`
- reusable external resources: use `record_resource_ref`

Recommended comment rule:

- keep comments as comment rows
- do not model first Kanban comments as child `record` rows

Recommended worklog rule:

- `started_at` and `ended_at` are canonical interval truth
- `seconds` is derived or cached
- do not allow more than one open interval for the same `(record_id, author_user_id)` pair

Recommended worklog index:

```
CREATE UNIQUE INDEX idx_record_worklog_one_open_interval
ON record_worklog(record_id, author_user_id)
WHERE ended_at IS NULL;
```

Normalized views

Widgets should consume normalized views, not raw storage assumptions.

Examples:

- simple record table: use only `record`
- lightweight Kanban: use `record` plus optional category metadata
- planning Kanban: use `record` plus sidecars and aggregates
- worklog dashboard: use `record_worklog` aggregates plus optional effort data

When arrays are needed in a SQL projection, prefer JSON text in v1.

Widget state

Use host `widgetState` for:

- panel open or closed state
- filters

- local display modes
- selected tabs
- persisted UI ergonomics

Do not use `widgetState` for:

- backend credentials
- durable business truth
- copies of backend config that the host already owns

8.4.4. Recommended Workflows

Keep the workflow obvious. Most mistakes come from mixing too many responsibilities.

Server-shaped official widget

Use this when backend truth is primary.

Recommended shape:

- host-owned auth and runtime state
- bridge as control plane
- Maud for structure
- Datastar fragments for server-owned repeated structure
- Datastar signals for shell and local UI state
- JS only for imperative browser edges

Good fits:

- record tables
- CRUD panels
- dashboards with server-owned structure
- Kanban-like surfaces with strong domain rules

Stable shell with local state

Use this when the shell is stable and most visible changes are local presentation state.

Recommended shape:

- small stable DOM shell
- frontend Datastar signals for display state
- optional backend seed data
- host `widgetState` for persisted ergonomics

Good fits:

- clocks
- counters
- compact dashboards
- notes with view toggles

Client-owned renderer

Use this when the frontend owns rendering and interaction rules.

Recommended shape:

- raw JSON snapshot or SSE
- JS or vendored framework renders the surface
- Datastar is optional
- host still owns auth and transport

Good fits:

- Chess
- client-heavy graphs
- legacy ports
- library-driven widgets

Foreign integration

Use this when truth mainly lives outside Lince.

Recommended shape:

- wrap or adapt the foreign system honestly
- keep Lince involvement narrow
- use bridge state for host integration, not to fake domain truth

Good fits:

- weather widgets
- Spotify control
- embedded apps
- imported HTML wrappers

Engine-driven surface

Use this when the browser runtime is the real surface.

Recommended shape:

- engine shapes the meaningful UI
- DOM shell is mostly chrome or fallback
- JS handles lifecycle and integration glue

Good fits:

- terminals
- canvas tools
- media tools
- WASM inspectors

Self-provisioning workflow

Use this when the widget must create or bind its own backing resources.

Recommended order:

1. Widget boots with bridge metadata and no fake assumptions.
2. Host decides whether the widget is configured, provisionable, or blocked.
3. Widget requests provisioning when needed.
4. Host creates or finds resources idempotently.
5. Host stores resolved linkage in `widgetState`.
6. Widget continues through `/contract`, `/stream`, and actions.

The board should support a `provisionable` state. A stream widget without `viewId` is not always broken.

Maud authoring conventions

For official `sand/` widgets written with Maud:

- prefer `@if`, `@else`, `@for`, and `@match` over building HTML strings by hand
- keep repeated structure on the server when the server already knows it
- keep fragments readable enough to become stable Datastar patch targets

Prefer shorthand when it makes the markup clearer:

- `.class-name { ... }`
- `#element-id { ... }`
- `p.class-name { ... }`
- `button.toolbar-btn { ... }`

Example:

```
@if items.is_empty() {
    .empty-state {
        "No items"
    }
} @else {
    ul.item-list {
        @for item in items {
            li.item-row data-id=(item.id) {
                .title { (item.title) }
            }
        }
    }
}
```

Avoid:

- large concatenated HTML strings when the server knows the structure
- verbose `div class="..."` markup when Maud shorthand is clearer
- pushing simple server-owned structure into client JS without a reason

9. Sandbox Website Mode

This document captures an opt-in Lince mode for serving a preset collection of workspaces as a public interactive surface.

The current name here is `sandbox website mode` because it matches the current discussion better than overloading the normal web host vocabulary.

The scope here is not "replace the normal website". The scope is:

- keep the old website
- add a second interactive published surface
- allow local browser play on top of a server-published preset
- keep the normal authenticated web/API host separate

9.1. What Is Agreed

- This is a host mode, not a widget trick.
- This should be compile-time gated behind a feature flag if binary bloat matters.
- Using the same source tree for two binaries is acceptable.
- The normal web/API binary stays broad.
- The sandbox binary stays narrow.
- The public mode starts from a server-published preset.
- The browser may keep a local overlay on top of that preset.
- There should be an explicit "reset to published preset" action.
- SSE is allowed for a narrow allowlisted set of bindings declared by the preset.
- Other Lince backend services and endpoints should not be available in the restricted mode.
- The old website remains.

Suggested deployment split:

- `sandbox.lince.social`: restricted preset mode
- `manas.lince.social`: normal API-only or authenticated web/API host

This split is better than trying to keep one public router broad while asking widgets to behave politely.

9.2. Binary And Feature Direction

The corrected direction is:

- feature flags, not just runtime configuration
- same repository
- separate binaries or feature-shaped entrypoints

That matters because runtime disablement does not meaningfully control binary composition on its own.

The project may still share most source code. The important part is that the restricted binary should not accidentally ship broad host behavior just because it is “turned off in config”.

9.3. Primary Model

The model is:

1. server publishes a preset
2. browser loads that preset as initial state
3. browser may alter that state locally
4. browser may reset back to the published preset
5. only explicitly allowed SSE bindings may reconnect automatically

That means the public mode is not ordinary board persistence. It is closer to a published scene plus a local overlay.

9.4. Persistence Model

Agreed direction:

- published preset from server
- local overlay in `localStorage`
- explicit reset action

This is better than partial persistence rules such as “save scroll but not widgets” because partial rules become incoherent quickly.

The browser should be able to answer three questions clearly:

- what is the published state?
- what is my local overlay?
- how do I return to the published state?

9.5. Required New Artifact

The current workspace archive shape is not enough because it only models one workspace.

The restricted mode needs a new artifact, conceptually something like:

- preset metadata
- active workspace
- ordered workspace list
- cards for each workspace

- package references or embedded package payloads
- allowlisted SSE bindings
- optional preset version

This should not be disguised as “just export one more workspace”.

The real object is a published board collection, not a single workspace.

For now the useful name is something like:

- `BoardCollectionArchive`
- or `SandboxPresetArchive`

The core requirement is:

- export all workspaces
- edit the result in normal mode
- re-package it
- publish it into sandbox mode

9.6. Required Route Policy

This is the main non-negotiable constraint.

The restriction must live in the host router, not in widget convention.

The public mode should expose only:

- the initial HTML/bootstrap route
- static assets
- local overlay mechanisms that never hit the server
- the allowlisted SSE route or routes
- whatever bridge/bootstrap support is required to keep the front-end viable without broad host APIs

The public mode should not expose:

- board state persistence routes
- generic package catalog routes
- generic DNA package routes
- server management routes
- terminal routes
- AI builder routes
- generic widget action routes
- arbitrary Lince integration endpoints

The public mode is only credible if the forbidden endpoints are absent or reject access.

9.7. Why The Normal Board Client Cannot Be Reused Unchanged

The issue is not only the widgets.

The current board client itself assumes broad host access. In current code, the board front-end persists board state through the host API, reads local package catalog endpoints, reads DNA package endpoints, and reads other broad host surfaces.

So the statement is:

- the normal board client may still be reused in pieces
- the normal board client cannot be reused unchanged

The unchanged client would try to call routes that the sandbox binary should not mount.

That means sandbox mode needs at least one of:

- a forked front-end entrypoint
- feature-gated client branches
- a stronger bootstrap contract that disables those code paths cleanly

This is not a theoretical complaint. It is a direct consequence of the current front-end assuming ordinary board persistence and package-management access.

9.8. Why Widget-Level Discipline Is Not Enough

Widgets are not trusted policy objects.

A preset may embed arbitrary HTML widgets. If those widgets run on the same origin and broad host routes are still mounted, then “please do not call those endpoints” is not a security model.

So the mode must assume:

- widgets may try to fetch what they can reach
- route policy is the real wall
- preset validation is still useful even after route sealing

This also answers the simplified argument:

- “if a widget assumes whole backend access in sandbox mode, it just will not work”

That is acceptable as a product outcome. It is not acceptable as the only enforcement layer.

The host still must refuse access.

9.9. SSE Policy

The SSE exception should be narrower than:

- “all views are readable if a widget asks”

Better rule:

- a preset declares its allowed live bindings
- only those bindings may open
- bindings are read-only
- bindings are not general backend capability tokens

The widget should not be able to widen the binding by editing its own host metadata or by manually changing `view_id`.

The host should validate the requested binding against the published preset.

That means:

- the preset fixes which stream binding belongs to which card or widget instance
- the public front-end does not expose UI for changing that binding
- manual client edits still do not widen access because host validation rejects them

This keeps the mode focused on published live surfaces instead of silently becoming a second authenticated web client.

9.10. Bridge Direction

Some host bridge or bootstrap support is still acceptable in sandbox mode if it only serves front-end viability.

Examples:

- initial board bootstrap
- instance metadata needed for layout or widget identity
- local-only overlay synchronization
- narrow read-only binding metadata

This is different from exposing general host services.

The bridge is acceptable when it helps the front-end remain interactive without turning the sandbox binary into a broad backend.

9.11. Edit, Build, Publish

The public mode and the editor should remain different concerns.

The stronger direction is:

- broad mode stays broad
- sandbox mode stays sealed
- publishing is a deliberate export step from broad mode into restricted mode

That implies a validator at publish time.

The validator should reject or strip cards that need forbidden capabilities.

Examples of cards that should be treated carefully:

- cards that depend on generic host actions
- cards that depend on server CRUD
- cards that assume unrestricted `/host` availability
- cards imported from DNA without restricted-mode compatibility

The point is not that every incompatible widget is forbidden everywhere. The point is:

- incompatible widgets may exist in broad mode
- incompatible widgets should not silently be published into sandbox mode

9.12. Blog And Website Content

Agreement here stays intentionally small for now.

The old website remains. This mode is additional.

Migrating home page widgets, blog widgets, and other content widgets may happen later. For now the important part is the host mode and its policy boundary.

This is the right priority because content conversion is not the hard problem. The hard problem is sealing the runtime surface while keeping the front-end viable.

9.13. Criticisms And Rejected Shortcuts

The following shortcuts should be rejected:

- “just let widgets promise not to call Lince endpoints”
- “permissions metadata alone is enough”
- “runtime disabled means no binary bloat”
- “single-workspace export is basically the same thing”
- “the normal board client can stay unchanged”
- “if an unrestricted widget breaks in sandbox mode, that alone is enough protection”
- “hiding the UI to change `view_id` is enough”

Reasons:

- widget HTML is executable code, not a trusted declaration
- metadata helps validation but does not replace router enforcement
- only compile-time structure meaningfully affects binary composition
- a published preset is structurally broader than one workspace archive
- the current board front-end already assumes broader host APIs
- broken widgets are acceptable, unauthorized host access is not
- SSE binding must be validated at the host, not only hidden in the UI

9.14. Criticisms Of The Remaining Ambition

The remaining ambition is reasonable, but a few costs should still be stated plainly.

This mode is good for:

- interactive published scenes
- playful local overlays
- personalized content surfaces
- narrow live dashboards

This mode is weaker for:

- frictionless reuse of every existing widget
- pretending the current board client needs no adaptation
- broad host capabilities under the same public binary

That does not make the mode wrong. It means the project should treat it as a deliberate second product surface, not as a nearly-free toggle.

9.15. Minimal Sound Slice

The smallest sound implementation is:

- feature-gated restricted binary
- new all-workspaces preset artifact
- sealed router with no generic host API surface
- one or more allowlisted read-only SSE bindings
- host validation of fixed widget-to-stream bindings
- local overlay in `localStorage`
- explicit reset to published preset
- publish-time preset validation
- front-end branches or a sandbox-specific entrypoint so the client does not try to call forbidden routes

Anything weaker than that risks turning the mode into an unrestricted host with polite language around it.

9.16. Kanban

This document is the Kanban-specific creation and implementation specification.

It is intentionally separate from the general web component specification. Read `web_components.typ` first for:

- host/runtime assumptions
- endpoint contracts
- runtime shape guidance

- sidecar-table strategy

9.16.1. Purpose

This specification defines the official Record Kanban for the web version.

The target is:

- Record-centric
- fed by a view SSE source
- persisted per widget through host `widgetState`
- authenticated by the host-owned web login
- implemented as a server-shaped board with rich local state

9.16.2. Target runtime shape

The official Kanban should be:

- server-shaped board with rich local state

Reason:

- server-owned board structure matters
- per-card UI state matters
- drag and drop matters
- resize matters
- connection/auth clarity matters
- local display ergonomics matter

A pure server-fragment surface is too rigid for the amount of local interaction state. A mostly local reactive shell pushes too much structure into the browser. A fully client-owned renderer makes JS the default rendering model. A foreign integration shape is not the center of gravity for a Lince-native Kanban. An engine-driven surface is the wrong fit for a DOM-first board.

9.16.3. Responsibility split

Normal center of gravity:

- Maud: main structural owner for repeated and server-owned fragments
- Datastar: stable shell state, derived presentation state, form drafts, loading indicators, and `widgetState` persistence glue
- JavaScript: drag/drop, resize, scroll restoration, offline worklog queueing, and other imperative behavior that Datastar should not be asked to fake

Normal transport split:

- HTML fragments from backend or official widget stream: primary
- signal patches: primary for shell and draft state
- raw JSON SSE: only as narrow fallback during transition

9.16.4. Runtime contract

Preferred runtime direction:

```
GET  /host/widgets/{instance_id}/stream
GET  /host/widgets/{instance_id}/contract
POST /host/widgets/{instance_id}/actions/{action}
```

Do not keep the final official Kanban limited to direct consumption of:

```
/host/integrations/servers/{server_id}/views/{view_id}/stream
```

The instance-aware runtime should:

- resolve the widget instance
- read `serverId`, `viewId`, `widgetState`, and stream state
- subscribe internally to backend data
- validate a record-centric contract
- render Maud fragments
- emit Datastar fragment and signal patches

9.16.5. Data contract

9.16.5.1. Required columns

The Kanban accepts record-centric view projections.

Minimum required core:

- `id`
- `quantity`
- `head`
- `body`

If those are not present, the widget must reject the input with a short mismatch state.

The widget does not need to enforce physical single-table origin. It should instead enforce a normalized record-centric contract.

9.16.5.2. Optional columns

If present, these should be consumed:

- `primary_category`
- `categories_json`
- `start_at`
- `end_at`
- `estimate_seconds`
- `actual_seconds`
- `assignee_ids_json`

- `assignee_names_json`
- `parent_id`
- `parent_head`
- `depth`
- `children_count`
- `children_json`
- `task_type`
- `comments_count`
- `last_comment_preview`
- `active_worklog_count`

If absent:

- the board still functions with core Record behavior
- dependent UI should hide cleanly
- the widget must not invent values

9.16.5.3. Current schema truth

Current stable Record truth is still:

- `id`
- `quantity`
- `head`
- `body`

That means:

- full CRUD for `head`, `body`, and `quantity` is already conceptually valid
- richer fields require view projection and recommended sidecar tables

9.16.5.4. Record-centric validation

The validation rule should be:

- the projection must expose the required Record core
- optional task metadata may come from joins, sidecars, aggregates, or relations
- the widget should prefer server-declared contract metadata when available
- if an official `/contract` route exists, it should be treated as the preferred validation source

The rule should not be:

- only one physical table is allowed

9.16.5.5. Normalized projection contract

Preferred normalized fields and types:

- `id`: integer
- `quantity`: integer

- `head`: text
- `body`: text
- `primary_category`: nullable text, used for compact badges when needed
- `categories_json`: JSON-encoded array of strings, representing all task categories in v1
- `start_at`: nullable UTC datetime text in `YYYY-MM-DDTHH:MM:SSZ`
- `end_at`: nullable UTC datetime text in `YYYY-MM-DDTHH:MM:SSZ`
- `estimate_seconds`: nullable integer
- `actual_seconds`: nullable integer derived from worklog intervals
- `assignee_ids_json`: JSON-encoded array of integer `app_user.id` values
- `assignee_names_json`: JSON-encoded array of display names aligned with assignee ids
- `parent_id`: nullable integer
- `parent_head`: nullable text
- `depth`: nullable integer
- `children_count`: nullable integer
- `children_json`: nullable JSON-encoded array of summary objects shaped as `{ "id": int, "head": string, "quantity": int, "task_type": string | null }`
- `task_type`: nullable enum of `epic`, `feature`, `task`, or `other`
- `comments_count`: nullable integer
- `last_comment_preview`: nullable text
- `active_worklog_count`: nullable integer counting all currently open intervals for the task across all users

Rules:

- arrays should prefer JSON text in v1 because SQL view projections are easier to keep stable that way
- category cardinality in v1 is many, not one
- `task_type` labels shown in the UI may be capitalized, but wire values should stay lowercase
- `actual_seconds` must not be edited directly; edit the intervals instead
- summary-level `actual_seconds` may lag behind currently running timers until the next heartbeat or detail refresh
- focused detail may add live client-side elapsed time on top of persisted or heartbeated interval totals
- if `/contract` is available, these fields should be declared there explicitly

Task inclusion rule:

- records appear in Kanban when `task_type` is present and valid
- preferred valid values are `epic`, `feature`, `task`, and `other`
- records with missing or empty `task_type` should not appear in the Kanban by default
- setting a valid `task_type` on a normal `record` is enough to make that record Kanban-ready
- categories are optional labels and are not required for Kanban inclusion

Category runtime rule:

- preserve category display casing and insertion order

- compare categories case-insensitively for filtering and matching

Task-type evolution rule:

- if the allowed task-type enum changes in a future release, migrate persisted `task.type` rows with an explicit SQL migration
- do not silently reinterpret unknown historic values at runtime
- prefer rewriting old values to new valid enum values or `NULL`, then updating the normalized Kanban views accordingly

Old-data migration policy:

- schema migration and old-data backfill are part of Lince, not widget-only behavior
- first implementation may ship with schema migrations only and no forced backfill of old records
- historic records without `task_type` remain valid `record` rows but do not appear in Kanban until a valid `task_type` is set
- the widget should not guess task types for legacy records on its own
- no bulk task-type action is required for v1

9.16.5.6. Recommended normalized view shape

The Kanban should normally consume a saved backend view whose query already projects the normalized record-centric shape.

The recommended direction is:

- one base saved view defines the dataset and normalized columns
- widget-local filter state is edited in the GUI and persisted in structured form in `widgetState`
- the internal widget stream service applies those filters by deriving SQL executed in SQLite from structured filter state
- the widget should not receive thousands of irrelevant rows just to filter them in JavaScript or Rust after the query runs

The widget should not treat user-edited SQL as the first implementation path.

Filter persistence scope:

- v1 persists structured filter clauses per widget instance in `widgetState`
- the host derives the effective SQL from those clauses when opening the Kanban stream
- the GUI should be populated from the structured clauses, not by reverse-parsing SQL
- do not mutate the saved base `view.query` whenever one Kanban instance changes filters
- do not store only a raw SQL filter string if the GUI must later reconstruct the applied filters

Future expansion:

- named presets or shared filtered boards are not required in v1
- if they become necessary later, add a dedicated persisted filter or preset model
- do not overload `view.query` with per-widget temporary filters

9.16.5.7. Current runtime flow

The current code does not synthesize a base view just because the user added a Kanban.

Current setup:

- the widget card must already point at a real `server_id` and `view_id`
- if `view_id` is missing, the widget is misconfigured
- the closest generic starting point is the normal view creation flow, whose default SQL is `SELECT * FROM record`
- Kanban itself only attaches to that existing base view
- the base view is the source dataset; Kanban layers runtime filters on top of it

When the widget loads:

- `/host/widgets/{instance_id}/contract` reads the card state and returns the source view metadata plus the persisted filter rows from `widgetState`
- `/host/widgets/{instance_id}/stream` reads the same card, loads the source view, validates that the stream is not a special non-SQL view, and builds a derived SQL query from the stored filter rows
- the host creates or updates a derived view named `__lince_web_kanban_{instance_id}`
- the widget subscribes to that derived view stream, not to the source view directly

When the user changes filters:

- the widget sends `apply-filters`
- `KanbanFilterService` persists the structured filter rows into `widgetState.filters`
- `filters_version` is incremented
- the widget reconnects, which causes the host to rebuild the derived query from the stored rows

When the user wants another Kanban:

- a second widget instance can point at the same source `view_id` and keep its own filters
- the derived view name includes the widget instance id, so the two Kanbans do not overwrite each other
- if the second board should use a different dataset, it needs a different source `view_id` or a different base view query
- there is no shared preset model in v1, so shared filtered boards are still just separate widget instances with duplicated filter state
- the host does not inject a separate Kanban projection query on top of the base view; it only wraps the base query with filter predicates and ordering

Readiness rule:

- a view is Kanban-ready when its snapshot exposes at least `id`, `quantity`, `head`, and `body`
- optional normalized columns can be missing; the widget hides the corresponding UI instead of inventing values
- if the snapshot columns do not satisfy the contract, the board renders a mismatch state instead of trying to coerce the data

9.16.5.8. Operator runbook

Use this flow when you need a Kanban board in practice:

1. Create or choose the source view.

- If you want a generic starting point, use the normal view creator. Its default SQL is `SELECT * FROM record`.
- If you want richer Kanban fields or joins, put that SQL in the source view itself.
- The Kanban runtime does not add its own projection layer later.

2. Point the widget at that view.

- Set the widget card `server_id`.
- Set the widget card `view_id`.
- The widget will not run without both values.

3. Open the widget.

- `/host/widgets/{instance_id}/contract` resolves the widget contract.
- `/host/widgets/{instance_id}/stream` loads the source view and derives a filtered SQL wrapper from the persisted filter rows.
- The host creates or reuses a derived view named `__lince_web_kanban_{instance_id}` and streams that derived view to the widget.

4. Apply filters from the UI.

- Filters are stored per widget instance in `widgetState`.
- Saving filters does not rewrite the source view query.
- After `apply-filters`, the widget reconnects so the host can rebuild the derived view query.

5. Decide what kind of second Kanban you need.

- If you want the same dataset with different filters, create another widget instance that points at the same source `view_id`.
- If you want a different dataset or a different projection, create another source view first and point the new widget at it.
- v1 has no shared filter preset model, so each Kanban instance owns its own filter state.

9.16.5.9. Example normalized SQL direction

The following is a recommended shape for a normalized Kanban view under the current sidecar plan:

```
SELECT
    r.id,
    r.quantity,
    r.head,
    r.body,
    json_extract(categories_ext.data_json, '$.categories[0]') AS
primary_category,
```

```
    json_extract(categories_ext.data_json, '$.categories') AS categories_json,
    json_extract(schedule_ext.data_json, '$.start_at') AS start_at,
    json_extract(schedule_ext.data_json, '$.end_at') AS end_at,
    json_extract(effort_ext.data_json, '$.estimate_seconds') AS
estimate_seconds,
    COALESCE(worklog_sum.actual_seconds, 0) AS actual_seconds,
    COALESCE(worklog_sum.active_worklog_count, 0) AS active_worklog_count,
    assignee_sum.assignee_ids_json,
    assignee_sum.assignee_names_json,
    parent_rel.parent_id,
    parent_record.head AS parent_head,
    child_sum.children_count,
    child_sum.children_json,
    json_extract(type_ext.data_json, '$.task_type') AS task_type,
    COALESCE(comment_sum.comments_count, 0) AS comments_count,
    comment_sum.last_comment_preview
FROM record r
LEFT JOIN record_extension categories_ext
    ON categories_ext.record_id = r.id
   AND categories_ext.namespace = 'task.categories'
LEFT JOIN record_extension schedule_ext
    ON schedule_ext.record_id = r.id
   AND schedule_ext.namespace = 'task.schedule'
LEFT JOIN record_extension effort_ext
    ON effort_ext.record_id = r.id
   AND effort_ext.namespace = 'task.effort'
LEFT JOIN record_extension type_ext
    ON type_ext.record_id = r.id
   AND type_ext.namespace = 'task.type'
LEFT JOIN (
    SELECT
        rl.record_id,
        json_group_array(rl.target_id) AS assignee_ids_json,
        json_group_array(u.name) AS assignee_names_json
    FROM record_link rl
    JOIN app_user u
      ON u.id = rl.target_id
    WHERE rl.link_type = 'assigned_to'
      AND rl.target_table = 'app_user'
    GROUP BY rl.record_id
) assignee_sum
    ON assignee_sum.record_id = r.id
LEFT JOIN (
    SELECT
        rl.record_id,
        MAX(rl.target_id) AS parent_id
    FROM record_link rl
    WHERE rl.link_type = 'parent'
      AND rl.target_table = 'record'
    GROUP BY rl.record_id
) parent_rel
    ON parent_rel.record_id = r.id
```

```
LEFT JOIN record parent_record
    ON parent_record.id = parent_rel.parent_id
LEFT JOIN (
    SELECT
        rl.target_id AS parent_id,
        COUNT(*) AS children_count,
        json_group_array(
            json_object(
                'id', child.id,
                'head', child.head,
                'quantity', child.quantity,
                'task_type', json_extract(child_type.data_json, '$.task_type')
            )
        ) AS children_json
    FROM record_link rl
    JOIN record child
      ON child.id = rl.record_id
    LEFT JOIN record_extension child_type
      ON child_type.record_id = child.id
     AND child_type.namespace = 'task.type'
    WHERE rl.link_type = 'parent'
      AND rl.target_table = 'record'
    GROUP BY rl.target_id
) child_sum
    ON child_sum.parent_id = r.id
LEFT JOIN (
    SELECT
        rw.record_id,
        CAST(SUM(
            CASE
                WHEN rw.ended_at IS NOT NULL
                    THEN COALESCE(rw.seconds, strftime('%s', rw.ended_at) -
strftime('%s', rw.started_at))
                ELSE COALESCE(rw.seconds, 0)
            END
        ) AS INTEGER) AS actual_seconds,
        SUM(CASE WHEN rw.ended_at IS NULL THEN 1 ELSE 0 END) AS
active_worklog_count
    FROM record_worklog rw
    GROUP BY rw.record_id
) worklog_sum
    ON worklog_sum.record_id = r.id
LEFT JOIN (
    SELECT
        rc.record_id,
        COUNT(*) AS comments_count,
        (
            SELECT rc2.body
            FROM record_comment rc2
            WHERE rc2.record_id = rc.record_id
              AND rc2.deleted_at IS NULL
            ORDER BY rc2.created_at DESC, rc2.id DESC
```

```
            LIMIT 1
        ) AS last_comment_preview
    FROM record_comment rc
    WHERE rc.deleted_at IS NULL
    GROUP BY rc.record_id
) comment_sum
    ON comment_sum.record_id = r.id
WHERE json_extract(type_ext.data_json, '$.task_type') IN ('epic', 'feature',
'task', 'other');
```

Critique of older query shapes:

- do not use `task.bucket`; use `task.categories`
- do not use scalar assignment metadata when assignees are real `app_user` relations
- do not use `MAX(body)` to mean latest comment preview; order by comment timestamps instead
- do not divide into `actual_hours` in storage or wire contracts; keep seconds canonical and format in the UI
- do not pretend hierarchy `depth` exists if the view is not actually deriving it

If hierarchy depth is not implemented, it should be `NULL` or omitted rather than faked.

9.16.5.10. Filter strategy

The Kanban should support structured GUI filters.

Preferred v1 model:

- a saved base view defines the broad dataset
- widget-local filters are stored in `widgetState`
- every filter row created in the GUI is combined with the others using logical `AND`
- multi-value filters inside one row use logical `OR`
- the internal widget stream service derives a filtered SQL wrapper around the base query
- the widget should reconnect the official stream after filter changes
- the base saved view SQL is not edited as raw text in the widget UI

Preferred v1 filter fields:

- `text_query`
- `categories_any_json`
- `assignee_ids_any_json`
- `quantities_json`
- `task_types_json`
- `only_with_open_worklog`

Filter semantics:

- `text_query`: case-insensitive match against `head` and optionally `body`
- `categories_any_json`: match if the record has any of the selected categories
- `assignee_ids_any_json`: match if the record has any of the selected assignees
- `quantities_json`: match explicit Kanban columns

- `task_types_json`: match selected task-type enum values
- `only_with_open_worklog`: restrict to records with `active_worklog_count > 0`

Default dataset rule:

- prefer `task_type IN ('epic', 'feature', 'task', 'other')` as the default inclusion rule for Kanban records
- do not require a redundant `"Task"` category when `task_type` already encodes that meaning
- use categories for project, team, or domain labels such as `project-1`, `project-2`, or `design`

Runtime readiness rule:

- any normal `record` can become Kanban-visible at runtime by setting a valid `task_type`
- no additional schema conversion is required for that record to enter the Kanban dataset
- categories remain optional and are mainly useful for filtering, grouping, and badges

Recommended derived SQL direction:

```sql
SELECT *
FROM (
    /* base kanban view query */
) base
WHERE 1 = 1
  AND (
      :text_query IS NULL
      OR lower(base.head) LIKE '%' || lower(:text_query) || '%'
      OR lower(base.body) LIKE '%' || lower(:text_query) || '%'
  )
  AND (
      :only_with_open_worklog = 0
      OR COALESCE(base.active_worklog_count, 0) > 0
  )
  /* quantities_json => base.quantity IN (...) */
  /* task_types_json => base.task_type IN (...) */
  /* categories_any_json => EXISTS (SELECT 1 FROM
json_each(COALESCE(base.categories_json, '[]')) c WHERE c.value IN (...)) */
  /* assignee_ids_any_json => EXISTS (SELECT 1 FROM
json_each(COALESCE(base.assignee_ids_json, '[]')) a WHERE CAST(a.value AS
INTEGER) IN (...)) */
ORDER BY base.quantity ASC, lower(base.head) ASC, base.id DESC;
```

Implementation rules:

- build the filter SQL on the host from structured filter clauses, not by accepting raw SQL fragments from the widget
- execute the derived filter query in SQLite
- do not stream a superset and then filter rows in Rust
- persist the filter clauses in `widgetState`
- if desired for debugging, the host may expose the effective SQL string in `/contract` or diagnostics, but it should remain derived data

- `apply-filters` should persist the new filters and increment a filters-version marker
- after `apply-filters`, the widget should reconnect `/host/widgets/{instance_id}/stream`
- the new stream connection should read the latest persisted filters and start a new filtered subscription generation
- use parameterized values when building the filtered query wrapper

9.16.6. Host services

The official Kanban should rely on explicit host-side services rather than pushing orchestration into the widget.

Preferred host service set:

- `KanbanContractService`
- `KanbanStreamService`
- `KanbanFilterService`
- `KanbanDetailService`
- `KanbanActionService`
- `KanbanWorklogService`
- `KanbanLivenessService`

`KanbanContractService`

- resolves widget instance metadata
- validates that the configured base view can produce the normalized Kanban contract
- returns `/host/widgets/{instance_id}/contract`
- may expose effective SQL for debugging, but not as the primary GUI persistence format

`KanbanStreamService`

- opens the instance-aware stream
- resolves the configured server and base view
- asks `KanbanFilterService` for the current filtered SQL wrapper
- subscribes to the filtered view stream
- emits Datastar fragments and signals or official event shapes for the widget

`KanbanFilterService`

- reads structured filter clauses from `widgetState`
- validates the filter rows and operators
- derives a parameterized SQL wrapper around the base view query
- increments and tracks `filters_version`
- is the authority for filter persistence and filter-to-SQL translation

`KanbanDetailService`

- loads the focused record detail by `record_id`
- returns the focused card HTML fragment and detail payload
- includes comments, child summaries, resources, assignees, and worklog interval detail

`KanbanActionService`

- handles semantic actions such as:
  - `create-record`
  - `update-record`
  - `move-record`
  - `delete-record`
  - `apply-filters`
- coordinates sidecar writes and relation writes when needed
- should own normal task-type and category CRUD as part of record create or update flows
- should not require a dedicated service per simple metadata field in v1
- should reject invalid hierarchy writes such as self-parent and obvious cycles
- last write wins is acceptable for concurrent metadata edits in v1

`KanbanWorklogService`

- handles `start-worklog` and `stop-worklog`
- enforces one open interval per `(record_id, author_user_id)`
- updates `last_heartbeat_at`
- computes or refreshes summary worklog totals when needed

`KanbanLivenessService`

- emits heartbeat or ping events for the official stream
- tracks stale and disconnected thresholds
- distinguishes paused updates from disabled transport

Service boundary decision:

- use one combined `KanbanActionService` for Record create, edit, delete, task-type edits, and category edits
- use one dedicated `KanbanFilterService` for filter validation, persistence, SQL derivation, and filter-version tracking
- do not create separate `TaskTypeService` or `CategoryService` in v1 unless they later acquire materially different business rules

9.16.7. Quantity mapping

Current Kanban column mapping must stay compatible:

- `0` => Backlog
- `-1` => Next
- `-2` => WIP
- `-3` => Review
- `1` => Done

This mapping is odd, but should remain for compatibility unless a broader migration is chosen.

9.16.8. Shell structure

The Kanban shell should be stable and Maud-owned.

Stable regions:

- widget header
- connection and action toolbar
- board viewport
- optional footer or status region

Preferred fragment patch targets:

- `#kanban-header-meta`
- `#kanban-toolbar-state`
- `#kanban-columns`
- `#kanban-empty-or-error`
- `#kanban-create-sheet`
- `#kanban-edit-sheet`

Preferred signal-owned state:

- connection badge state
- paused, running, reconnecting, disconnected, and locked state
- live connection enabled or disabled state
- query panel open state
- global body display mode
- per-card body display mode
- focused record id
- focus markdown mode
- collapsed columns
- column widths
- active sheet or drawer
- filter drafts
- active filter summary state
- create, edit, and focus-action drafts
- local validation and loading state that is safe to persist or keep local

9.16.9. Technology ownership

9.16.9.1. Maud

Maud should own:

- shell
- loading state
- locked state
- mismatch state

- compact error state
- header and board structure
- columns and cards
- create, edit, and focus surfaces
- repeated filter, comment, child, and resource rows
- empty states

9.16.9.2. Datastar

Datastar should own:

- local UI signals
- connection/auth/runtime indicators
- query open state
- collapse state
- width and visibility styles driven from signals
- body mode state
- form draft state through `data-bind`
- focus shell state
- loading and pending indicators
- button labels, disabled state, and small stable-shell reactivity
- signal-driven persistence glue into `widgetState`
- host bridge hydration into signals

9.16.9.3. JavaScript

JavaScript should own:

- drag and drop between columns
- pointer-driven column resize
- scroll restoration and focus positioning
- offline worklog stop queue and heartbeat loop
- minimal interop for markdown hydration or Datastar patch transport while the backend still needs it
- optimistic move choreography where needed

9.16.10. Persistence

Persist runtime ergonomics in `widgetState` through the host:

- column widths
- collapsed columns
- global body display mode
- per-card body display mode
- focused card identifier when applicable
- local filters

- stream paused preference

Do not persist business truth in `widgetState`.

Do not persist widget ergonomics in sidecar business tables.

Persistence should stay sparse:

- persist global defaults first
- persist per-card overrides only when they differ from the default

This remains the preferred model even if board state is later split into multiple files or directories.

9.16.11. Card content model

Core card content:

- `head` as title
- `body` as description
- `quantity` as Kanban status

Optional content extensions:

- categories
- dates
- estimate
- actual work
- assignees
- hierarchy
- comments
- task type
- parent label

Important distinction:

- task categories are task metadata used for grouping or categorization inside Kanban semantics
- bucket image references are external asset references to files or objects stored in a bucket
- these are different concepts and should not reuse the same storage model

Body display modes per card:

- hidden
- excerpt
- full

Focused card mode:

- a selected card may take over the component as a dedicated focus surface with a way back to the board
- this should be treated as focus mode, not only as another inline body mode

There should also be a global control to switch body display mode across the board.

9.16.12. Frontend plan

9.16.12.1. Task type and category CRUD

Preferred surfaces:

- create sheet for new records
- edit sheet for existing records
- focused card mode for richer editing context

Preferred controls:

- `task_type`: single-select control with values `epic`, `feature`, `task`, `other`
- `categories`: multi-value chip or token input

Preferred frontend behavior:

- create sheet should require `head` and allow optional `body`
- create sheet should default `task_type` to empty
- create sheet should allow setting `task_type` immediately so the new record can become Kanban-ready at creation time
- edit sheet should allow changing `task_type`
- edit sheet should allow adding and removing categories
- category chips should preserve insertion order in v1 and do not need sorting
- if `task_type` is cleared, show a short warning that the record will disappear from the Kanban after save
- category edits should update badges and category filter availability on the next stream refresh
- when all categories are removed, the preferred write behavior is to remove the `task.categories` sidecar row instead of storing an empty array

Preferred action usage:

- use `create-record` for new records with optional `task_type` and `categories`
- use `update-record` for task-type and category edits
- do not add `set-task-type` or `set-categories` actions in v1

9.16.12.2. Filter builder GUI

Preferred placement:

- a toolbar button opens a dedicated filter drawer or sheet
- active filters should also be visible as compact chips in the header or toolbar

Preferred v1 filter-builder model:

- each row in the GUI is an `AND` clause
- multi-value selection inside a row is an `OR`

- the filter builder edits structured filter rows, not SQL text
- filters are per widget instance and are not shareable in v1
- full text search should use case-insensitive `LIKE` in v1; FTS is not required

Preferred filter-builder row types:

- text contains
- categories any-of
- assignees any-of
- task types any-of
- quantities any-of
- only with open worklog

Preferred frontend behavior:

- editing filters does not mutate the saved base `view.query`
- applying filters persists the structured rows into `widgetState`
- after save, the widget calls `apply-filters` and reconnects the official stream
- the header should show the currently active filters in human-readable form
- the drawer should be able to reconstruct itself from persisted structured filter rows
- provide a clear-all action
- provide per-row remove actions

Preferred validation:

- do not allow empty filter rows to persist
- do not allow unknown filter fields or operators
- normalize multi-value rows to unique values before persisting

9.16.13. CRUD model

9.16.13.1. Create

Create flow should support:

- creating the core `record`
- optionally writing `record_extension`
- optionally writing `record_link`

Prefer a semantic action endpoint when available, but do not block the first implementation on it. Generic writes for `record`, `record_extension`, and `record_link` are acceptable for v1.

9.16.13.2. Edit

Edit flow should support:

- editing `head`
- editing `body`
- editing supported optional metadata when present

Task metadata CRUD in v1 should be handled through the same create and update surfaces as the core Record.

Recommended v1 rule:

- `task_type` and `categories` are edited through the normal Kanban create or edit sheet
- do not create a separate dedicated CRUD surface only for task type or categories in the first implementation
- clearing `task_type` is allowed and removes the record from Kanban visibility after stream refresh
- clearing all categories is allowed and does not remove the record from Kanban visibility

9.16.13.3. Move

Move flow should:

- patch `record.quantity`
- update UI optimistically if useful
- reconcile against server truth

9.16.13.4. Delete

Delete flow should delete the core `record`.

If sidecar tables are implemented with cascading deletes, that should remove:

- `record_extension`
- `record_link`
- `record_comment`
- `record_worklog`

9.16.14. Optional feature mapping

Use the recommended sidecar strategy from `web_components.typ`.

Mapping:

- task categories => `record_extension` namespace `task.categories`
- start/end date => `record_extension` namespace `task.schedule`
- estimate => `record_extension` namespace `task.effort`
- task type => `record_extension` namespace `task.type`
- assignees => `record_link` to `app_user`, many-to-many using `assigned_to`
- hierarchy => `record_link` using `parent`
- comments => `record_comment`
- accumulated real work => `record_worklog`

Bucket image references:

- if records need queryable reusable bucket image references, use `record_resource_ref`
- do not make body parsing the canonical attachment model

- do not overload `task.categories` with bucket-object references
- use `provider = "bucket"` and `resource_kind = "image"` in v1
- keep these references as detail-oriented data, not mandatory board-summary payload

9.16.15. Proposed SQLite migrations

The Kanban plan assumes the following tables and indexes exist.

Core sidecars:

```sql
CREATE TABLE record_extension (
    id INTEGER PRIMARY KEY,
    record_id INTEGER NOT NULL REFERENCES record(id) ON DELETE CASCADE,
    namespace TEXT NOT NULL,
    version INTEGER NOT NULL DEFAULT 1,
    data_json TEXT NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(record_id, namespace),
    CHECK (json_valid(data_json))
);

CREATE INDEX idx_record_extension_namespace_record
ON record_extension(namespace, record_id);

CREATE TABLE record_link (
    id INTEGER PRIMARY KEY,
    record_id INTEGER NOT NULL REFERENCES record(id) ON DELETE CASCADE,
    link_type TEXT NOT NULL,
    target_table TEXT NOT NULL,
    target_id INTEGER NOT NULL,
    position REAL,
    data_json TEXT,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(record_id, link_type, target_table, target_id),
    CHECK (data_json IS NULL OR json_valid(data_json))
);

CREATE INDEX idx_record_link_record_type
ON record_link(record_id, link_type, target_table);

CREATE INDEX idx_record_link_target_type
ON record_link(target_table, target_id, link_type);

CREATE TABLE record_comment (
    id INTEGER PRIMARY KEY,
    record_id INTEGER NOT NULL REFERENCES record(id) ON DELETE CASCADE,
    author_user_id INTEGER REFERENCES app_user(id) ON DELETE SET NULL,
    body TEXT NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```sql
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    deleted_at DATETIME
);

CREATE INDEX idx_record_comment_record_created
ON record_comment(record_id, created_at DESC);

CREATE TABLE record_worklog (
    id INTEGER PRIMARY KEY,
    record_id INTEGER NOT NULL REFERENCES record(id) ON DELETE CASCADE,
    author_user_id INTEGER NOT NULL REFERENCES app_user(id) ON DELETE CASCADE,
    started_at DATETIME NOT NULL,
    ended_at DATETIME,
    last_heartbeat_at DATETIME,
    seconds REAL,
    note TEXT,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_record_worklog_record_started
ON record_worklog(record_id, started_at DESC);

CREATE INDEX idx_record_worklog_author_started
ON record_worklog(author_user_id, started_at DESC);

CREATE UNIQUE INDEX idx_record_worklog_one_open_interval
ON record_worklog(record_id, author_user_id)
WHERE ended_at IS NULL;

CREATE TABLE record_resource_ref (
    id INTEGER PRIMARY KEY,
    record_id INTEGER NOT NULL REFERENCES record(id) ON DELETE CASCADE,
    provider TEXT NOT NULL,
    resource_kind TEXT NOT NULL,
    resource_path TEXT NOT NULL,
    title TEXT,
    position REAL,
    data_json TEXT,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(record_id, provider, resource_path),
    CHECK (data_json IS NULL OR json_valid(data_json))
);

CREATE INDEX idx_record_resource_ref_record_position
ON record_resource_ref(record_id, position, id);
```

Recommended namespace payloads:

```
task.categories => { "categories": ["project-1", "design"] }
task.schedule => { "start_at": "2026-03-28T12:00:00Z", "end_at": null }
```

```
task.effort => { "estimate_seconds": 14400 }
task.type => { "task_type": "feature" }
```

9.16.16. Worklog model

Worklog should use a dedicated table, not sidecar JSON arrays.

Recommended model:

- one row per interval
- intervals are per `app_user`
- many different users may have active intervals on the same record at the same time
- one user should not have more than one open interval for the same record at the same time
- `actual` is derived from interval sums
- `started_at` and `ended_at` are canonical interval truth
- `seconds` is derived or cached only, never a competing source of truth

Current-user resolution for worklog authorship in v1:

- the host resolves a stable current `app_user.id` and caches it in widget runtime state under `kanban_runtime.current_user_id`
- if the server session exposes a stable `username_hint`, prefer matching that to `app_user.username`
- if no username hint can be resolved in the current host runtime, fall back deterministically to the smallest available `app_user.id`
- widgets do not send arbitrary `author_user_id` values for normal start or stop worklog flows

Canonical interval fields:

- `record_id`
- `author_user_id`
- `started_at`
- `ended_at`
- `last_heartbeat_at`
- optional `note`

Running-time behavior:

- frontend shows live elapsed time while an interval is active
- backend writes should not happen every second
- coarse heartbeat writes such as every 5 minutes are acceptable for crash recovery

Enforcement:

- enforce the one-open-interval-per-`(record_id, author_user_id)` rule in both service logic and database indexing
- treat `ended_at IS NULL` as the definition of an open interval
- do not treat `seconds` as a separately editable source of truth

Offline stop queue storage:

- persist queued offline stops in host `widgetState`, namespaced under a dedicated key such as `pending_worklog_stops`
- queue entries should include only the known interval id and intended `ended_at`
- replay on reconnect and prune successful entries immediately
- do not promise full offline timer start in v1

Offline stop behavior:

- queue offline stop only for a known active interval that was already started online
- capture the intended `ended_at` locally and replay when connectivity returns
- full offline start is not required for the first implementation

9.16.17. Summary and detail loading

The board should prefer a summary projection for normal board rendering.

Preferred board payload:

- core Record fields
- category, dates, task type, hierarchy summary
- assignee summaries
- comments count or preview
- worklog summary

Preferred detail loading:

- full comments
- full attachment or bucket-image references
- richer edit surfaces
- any heavy card detail not needed for normal board rendering

This means:

- stream summary data for the board
- fetch or patch focused detail lazily when needed

Preferred focused-detail action:

- `POST /host/widgets/{instance_id}/actions/load-record-detail`

Preferred focused-detail request:

```
{
  "record_id": 123
}
```

Preferred focused-detail response:

```
{
  "record_id": 123,
  "target": "#kanban-focus-card",
  "mode": "replace",
```

```
  "html": "<section>...</section>",
  "signals": {
    "focusedRecordId": 123,
    "focusMode": true
  },
  "detail": {
    "primary_category": "project-1",
    "categories": ["project-1", "design"],
    "assignees": [
      { "id": 7, "name": "Ana" }
    ],
    "comments": [
      {
        "id": 88,
        "author_user_id": 7,
        "body": "Full comment body",
        "created_at": "2026-03-28T12:00:00Z",
        "updated_at": "2026-03-28T12:00:00Z"
      }
    ],
    "children": [
      {
        "id": 456,
        "head": "Child task"
      }
    ],
    "resources": [
      {
        "id": 12,
        "provider": "bucket",
        "resource_kind": "image",
        "resource_path": "bucket/path/image.png",
        "title": "Screenshot"
      }
    ],
    "worklog": {
      "actual_seconds": 14400,
      "active_worklog_count": 2,
      "intervals": [
        {
          "id": 991,
          "author_user_id": 7,
          "started_at": "2026-03-28T10:00:00Z",
          "ended_at": null,
          "last_heartbeat_at": "2026-03-28T12:00:00Z",
          "seconds": 7200,
          "note": null
        }
      ]
    }
  }
}
```

Response rules:

- `html` is the primary render surface for focus mode
- `detail` exists to support imperative interactions and small local computations without reparsing DOM
- `signals` should remain minimal and only express focus-state transitions

9.16.18. Action contract

For semantic Kanban actions, prefer one common response envelope.

Preferred success envelope:

```
{
  "ok": true,
  "action": "move-record",
  "message": "Record moved.",
  "record_id": 123,
  "await_stream_refresh": true,
  "detail": null
}
```

Preferred failure envelope:

```
{
  "ok": false,
  "action": "move-record",
  "message": "Unable to move record.",
  "code": "validation_error"
}
```

Action payloads:

`create-record`

Request:

```
{
  "record": {
    "head": "New task",
    "body": "Longer description",
    "quantity": 0
  },
  "task_type": "task",
  "categories": ["project-1", "design"],
  "start_at": null,
  "end_at": null,
  "estimate_seconds": 14400,
  "assignee_ids": [7, 9],
  "parent_id": null
}
```

Response:

```
{
  "ok": true,
  "action": "create-record",
  "message": "Record created.",
  "record_id": 123,
  "await_stream_refresh": true,
  "detail": {
    "focus_record_id": 123
  }
}
```

`update-record`

Request:

```
{
  "record_id": 123,
  "head": "Updated title",
  "body": "Updated body",
  "quantity": -1,
  "task_type": "feature",
  "categories": ["project-1"],
  "start_at": "2026-03-28T12:00:00Z",
  "end_at": null,
  "estimate_seconds": 21600,
  "assignee_ids": [7],
  "parent_id": 45
}
```

Response:

```
{
  "ok": true,
  "action": "update-record",
  "message": "Record updated.",
  "record_id": 123,
  "await_stream_refresh": true,
  "detail": null
}
```

`move-record`

Request:

```
{
  "record_id": 123,
  "quantity": -2
}
```

Response:

```
{
  "ok": true,
  "action": "move-record",
  "message": "Record moved.",
  "record_id": 123,
```

```
  "await_stream_refresh": true,
  "detail": {
    "quantity": -2
  }
}
```

delete-record

Request:

```
{
  "record_id": 123
}
```

Response:

```
{
  "ok": true,
  "action": "delete-record",
  "message": "Record deleted.",
  "record_id": 123,
  "await_stream_refresh": true,
  "detail": null
}
```

start-worklog

Request:

```
{
  "record_id": 123,
  "note": null
}
```

Response:

```
{
  "ok": true,
  "action": "start-worklog",
  "message": "Worklog started.",
  "record_id": 123,
  "await_stream_refresh": true,
  "detail": {
    "interval": {
      "id": 991,
      "author_user_id": 7,
      "started_at": "2026-03-28T10:00:00Z",
      "ended_at": null,
      "last_heartbeat_at": "2026-03-28T10:00:00Z",
      "seconds": 0,
      "note": null
    }
  }
}
```

stop-worklog

Request:

```
{
  "record_id": 123,
  "interval_id": 991,
  "ended_at": "2026-03-28T12:00:00Z"
}
```

Response:

```
{
  "ok": true,
  "action": "stop-worklog",
  "message": "Worklog stopped.",
  "record_id": 123,
  "await_stream_refresh": true,
  "detail": {
    "interval_id": 991,
    "actual_seconds": 14400
  }
}
```

`apply-filters`

Request:

```
{
  "filters": [
    { "field": "categories_any_json", "operator": "any_of", "value":
["project-1"] },
    { "field": "assignee_ids_any_json", "operator": "any_of", "value": [7,
9] },
    { "field": "task_types_json", "operator": "any_of", "value": ["feature",
"task"] },
    { "field": "quantities_json", "operator": "any_of", "value": [0, -1,
-2] },
    { "field": "text_query", "operator": "contains", "value": "search term" }
  ]
}
```

Response:

```
{
  "ok": true,
  "action": "apply-filters",
  "message": "Filters applied.",
  "record_id": null,
  "await_stream_refresh": true,
  "detail": {
    "filters_version": 2
  }
}
```

`apply-filters` rules:

- the response means the filter state was persisted, not that the old stream was mutated in place
- the widget should open a new stream connection after success
- the new stream must be backed by SQL-filtered results, not Rust-side row filtering

Preferred focused-detail responsibilities:

- render the focused card surface into `#kanban-focus-card`
- include full comments
- include bucket image references
- include child task summaries using `children_json`
- preserve board scroll and column scroll positions while focus mode is open

9.16.19. Connection and auth behavior

The board must clearly communicate:

- whether SSE is connected
- whether stream delivery is paused
- whether live connection is disabled
- whether auth has fallen
- whether host configuration is missing

Rules:

- do not implement widget-local login UI for the standard host-mediated flow
- use the general web login
- if auth is missing, show locked or waiting state
- if host config is missing, show misconfigured state
- if `meta.streams.enabled` is `false`, stop reconnecting until re-enabled

The runtime should separate:

- paused updates
- live transport enabled or disabled
- disconnected or stale connection

Heartbeat or explicit liveness signaling is preferred over purely heuristic last-message guesses.

Preferred liveness timing:

- emit a stream heartbeat or ping every 15 seconds when no snapshot has been sent
- mark the connection stale after 45 seconds without snapshot or heartbeat
- mark the connection reconnecting or disconnected after 90 seconds without snapshot or heartbeat

Required controls:

- pause receiving SSE data
- toggle connection or reception behavior if the runtime supports it

9.16.20. Layout and scrolling

The board should:

- occupy the full height of the component
- allow horizontal scrolling when columns overflow
- allow internal component scrolling like a page
- allow individual column scrolling

Column behavior should support:

- per-column scroll
- resize by dragging
- collapse or minimize

9.16.21. Hierarchy behavior

Hierarchy is metadata-first in the first implementation.

Rules:

- child and parent may exist in different columns
- moving a parent does not move children
- indenting is not required
- show the parent relation as a clickable label or chip inside the card in every body mode
- the same parent link should remain clickable in focus mode
- focused mode should also show child summaries with `head` only and allow clicking into another focused card
- task type color is separate from hierarchy relation
- self-parent is invalid
- parent links that would create a cycle are invalid

Grouping subtrees may come later, but is not required for the first implementation.

Meaning of metadata-first rather than tree-first:

- store and expose parent-child relations as normal metadata
- show parent and child navigation hints in cards and focus mode
- do not make recursive layout, nested rendering, subtree drag, or cascade moves part of the first implementation
- do not compute or rely on deep hierarchy behavior unless a later phase explicitly adds it

9.16.22. Comments behavior

Comments should use a summary-versus-detail model.

Board-level behavior:

- use `comments_count`
- optionally show one preview

Focused-detail behavior:

- load or patch full comments lazily
- do not stream full comment threads into the main board payload by default
- comments should remain in `record_comment`, not child `record` rows, for the first implementation
- focused mode may show full comment bodies and timestamps

9.16.23. Permissions and actions

Preferred v1 permission model:

- `read_view_stream` for summary board streaming
- `write_records` for core Record create, edit, move, and delete
- `write_table` for `record_extension`, `record_link`, `record_comment`, `record_worklog`, and `record_resource_ref`

Do not require finer-grained per-feature permissions in v1 unless the host already exposes them.

Preferred v1 semantic actions:

- `load-record-detail`
- `create-record`
- `update-record`
- `move-record`
- `delete-record`
- `start-worklog`
- `stop-worklog`
- `apply-filters`

Generic table writes remain acceptable when semantic actions do not yet exist.

9.16.24. UX states

Required explicit states:

- loading
- empty
- mismatch
- locked/auth missing
- misconfigured
- live
- paused
- focus mode
- reconnecting or disconnected
- stale if heartbeat or liveness expires

9.16.25. Patch strategy

HTML fragment patches are preferred for:

- columns
- card lists
- empty/error panels
- create/edit/focus surfaces
- repeated comments, children, resources, and option lists
- structural board updates

Signal patches are preferred for:

- connection state
- auth and lock state
- pause state
- sheet and focus open state
- body mode state
- collapse state
- width metadata
- header/toolbar status
- draft defaults and loading indicators

Rules:

- do not patch the entire document
- do not stream script tags as normal update fragments
- patch stable inner targets
- keep repeated collections fragment-owned rather than inventing client-side list rendering in Datastar
- keep JS loaded once, then patch HTML and signals around it

9.16.26. Datastar feature selection

Use Datastar aggressively for:

- `data-signals` to hold root Kanban runtime state, per-sheet drafts, and persisted UI ergonomics
- `data-bind` for filter, create, edit, and focus-action inputs
- `data-computed` for derived labels, disabled states, active-filter booleans, and lane/body-mode presentation state
- `data-show`, `data-class`, `data-style`, `data-text`, and `data-attr` for visibility, classes, widths, text, and button attributes
- `data-on` for open, close, toggle, clear, save, and debounced text-input behavior
- `data-indicator` for contract, detail, and action loading states
- `data-effect` and `data-on-signal-patch` for sparse `widgetState` persistence through the host bridge

- `data-ignore-morph` only on narrow imperative islands such as an actively dragged card, a resizing lane edge, or a DOM subtree the user is editing
- `data-ref` only as a helper for the small remaining imperative code

Do not use Datastar as:

- the renderer for large repeated board structures; keep columns, cards, comments, and resources as Maud fragments
- the drag-and-drop or resize engine
- the authoritative store of business truth
- a place to put side effects inside `data-computed`; the reference explicitly says not to do that
- the primary heartbeat or offline queue worker; `data-on-interval` is acceptable for cosmetic timers, not for critical worklog delivery guarantees
- a reason to overuse `data-init` on frequently patched nodes; it should stay on stable bootstrap points because patched attributes re-run initialization
- a production dependency on `data-json-signals`; keep that for debugging only

9.16.27. Functional checklist

Must support:

- host `widgetState` persistence
- SSE view data consumption
- record-centric validation
- CRUD on Record
- quantity-based column mapping
- full-height component layout
- horizontal board scroll
- internal component scroll
- per-column scroll
- per-column resize
- column collapse
- per-card body mode
- global body mode
- focused card mode
- pause/reconnect/live state visibility
- backend/auth failure visibility
- delete cards

Schema-dependent advanced features:

- comments
- hierarchy
- assignees through `app_user`
- category metadata
- dates

- estimate and actual work

9.16.28. Implementation step checklist

Each step below is intended to be testable before moving to the next one.

Current status convention:

- `implemented in code`: the required code path exists and compiles
- `runtime verification pending`: the feature still needs authenticated browser testing in the real widget flow
- `final polish pending`: the architecture is in place, but some UX cleanup or simplification still remains
- [x] Step 1: Freeze the implementation contract

Requirements:

- the normalized Kanban projection fields are frozen
- the task-type enum is frozen
- the semantic action payloads and response envelopes are frozen
- the `/contract` payload is frozen
- the SQL-side filter model is frozen
- the liveness thresholds are frozen

Step 1 passes when:

- no open architecture questions remain in this document for core Kanban behavior
- implementation can begin without redefining data shapes
- [x] Step 2: Add database migrations

Requirements:

- add `record_extension`
- add `record_link`
- add `record_comment`
- add `record_worklog`
- add `record_resource_ref`
- add the required indexes and unique constraints

Step 2 passes when:

- migrations apply cleanly to a fresh database
- migrations apply cleanly to an existing database
- the one-open-worklog invariant is enforced at the DB level
- [x] Step 3: Validate data-layer behavior

Requirements:

- generic CRUD works for the new tables
- `app_user` relations work for assignees
- `task.type` and `task.categories` writes behave as specified
- removing all categories removes the `task.categories` sidecar row
- hierarchy writes reject self-parent and obvious cycles

Step 3 passes when:

- manual table writes and reads produce the expected normalized data
- invalid hierarchy writes are rejected
- [x] Step 4: Implement host contract service

Requirements:

- implement `KanbanContractService`
- `/host/widgets/{instance_id}/contract` resolves widget instance metadata
- the contract route exposes required and optional fields
- the contract route exposes permissions, liveness thresholds, and relation declarations

Step 4 passes when:

- a configured widget instance returns a stable contract response
- a misconfigured widget instance returns a compact contract error
- [x] Step 5: Implement SQL-side filter service

Requirements:

- implement `KanbanFilterService`
- filter rows are persisted in structured form in `widgetState`
- the service derives parameterized SQL executed in SQLite
- the service increments and tracks `filters_version`
- no Rust-side row filtering is used after the query runs

Step 5 passes when:

- applying filters changes the effective SQL-backed dataset
- the GUI can be reconstructed from persisted filter rows
- [x] Step 6: Implement instance-aware stream service

Requirements:

- implement `KanbanStreamService`
- `/host/widgets/{instance_id}/stream` resolves server, base view, filters, and liveness state
- the stream uses the filtered SQL wrapper from `KanbanFilterService`
- heartbeat and stale/disconnected timing are emitted according to the spec

Step 6 passes when:

- the widget can connect through the instance-aware stream

- filter changes cause a new filtered subscription generation
- stale and disconnected states can be observed deterministically
- [x] Step 7: Implement semantic action services

Requirements:

- implement `KanbanActionService`
- implement `KanbanWorklogService`
- implement `create-record`
- implement `update-record`
- implement `move-record`
- implement `delete-record`
- implement `start-worklog`
- implement `stop-worklog`
- implement `apply-filters`

Step 7 passes when:

- every action returns the specified response envelope
- task-type and category CRUD works through `create-record` and `update-record`
- worklog invariants hold through the service layer
- [x] Step 8: Implement Maud shell and fragments

Requirements:

- implement the stable shell in Maud
- use Maud-native `@if` and `@for` where structure is server-owned
- keep Maud markup idiomatic with shorthand such as `.class-name {}` and `p.class-name {}`
- add stable fragment targets for header, toolbar, board, create sheet, edit sheet, and focus sheet
- implement loading, locked, mismatch, empty, and misconfigured states in Maud

Step 8 passes when:

- the Kanban can render from server-owned Maud fragments without client-built HTML strings for the main board
- the fragment targets match the contract in this document
- [x] Step 9: Implement signal hydration and widget-state persistence

Requirements:

- signal-patch local runtime state into the widget
- persist sparse UI state in `widgetState`
- persist widths, collapsed columns, body modes, focus state, and structured filters
- keep host state and local ergonomics separate from business truth

Step 9 passes when:

- reloads preserve the intended UI ergonomics
- no redundant dense per-card state is written when defaults are sufficient
- implementation note: persistence exists in code today, but the remaining manual DOM sync in `script.rs` should still be collapsed into Datastar signal ownership
- [x] Step 10: Implement core board interactions

Requirements:

- drag and drop between quantity columns
- optimistic move with rollback on failure
- full-height board layout
- board scroll and per-column scroll
- column resize
- column collapse
- global and per-card body modes

Step 10 passes when:

- the current board ergonomics are preserved or improved
- `move-record` replaces direct generic Record patching from the widget
- implementation note: drag resize now uses the column side borders rather than dedicated width buttons
- [x] Step 11: Implement create and edit surfaces

Requirements:

- create sheet supports `head`, optional `body`, optional `task_type`, optional `categories`
- create sheet defaults `task_type` to empty
- edit sheet supports task-type and category CRUD
- category chips preserve insertion order
- clearing `task_type` warns that the record will disappear from Kanban after save

Step 11 passes when:

- a normal record can become Kanban-ready by setting `task_type`
- clearing all categories does not remove the record from Kanban
- clearing `task_type` removes it from Kanban after refresh
- [x] Step 12: Implement filter builder GUI

Requirements:

- toolbar button opens filter drawer or sheet
- each row is `AND`
- multi-value within a row is `OR`
- row types include text, categories, assignees, task types, quantities, and open-worklog
- active filters are shown as compact chips

- clear-all and per-row removal exist

Step 12 passes when:

- the GUI can round-trip persisted filters without raw SQL editing
- applying filters reconnects the stream and yields SQL-filtered results
- implementation note: filter opening should be local and immediate because options are prefetched with the contract; runtime verification is still required on a fresh imported widget instance
- [x] Step 13: Implement focus mode detail loading

Requirements:

- implement `KanbanDetailService`
- implement `load-record-detail`
- focus mode renders into the dedicated focus target
- preserve board and column scroll positions while focus mode is open
- parent links are clickable in cards and focus mode
- child summaries in focus mode show `head` and allow navigation

Step 13 passes when:

- focus mode can open and close without losing board context
- parent and child navigation behave as specified
- implementation note: the focus path is implemented in code, but it still requires runtime verification on a fresh imported widget instance
- [x] Step 14: Implement relational and aggregate task detail

Requirements:

- comments summary on the board
- full comments in focus mode
- assignees through `app_user`
- task categories and badges
- worklog intervals, live timer display, heartbeat, and offline stop queue

Step 14 passes when:

- comments, assignees, task type, categories, and worklog are all visible and editable through the intended surfaces
- running worklog state stays consistent across reconnects
- implementation note: comments remain `record_comment`, not `record`; resources remain separate relations; live reconnect behavior still needs authenticated browser verification
- [ ] Step 15: Implement external resource detail and final polish

Requirements:

- bucket image references through `record_resource_ref`

- focus-mode resource rendering
- move remaining stable-shell UI state from imperative DOM mutation to Datastar signals and bindings
- remove JS string-built sheets, forms, chips, and action panels where Maud fragments plus Datastar bindings are sufficient
- final connection and auth UX polish
- review patch granularity and remove obsolete client-side SSE assumptions

Step 15 passes when:

- imperative JS in the widget is limited mainly to drag/drop, resize, scroll restoration, offline worklog queueing, and thin transport interop
- the widget no longer depends on raw snapshot parsing for its primary runtime
- the end-to-end Kanban matches this specification closely enough that only polish tasks remain

Current implementation summary:

- implemented in code:
  - sidecar migrations and indexes
  - record-centric contract route
  - SQL-side derived filtering
  - instance-aware Kanban stream
  - semantic Kanban actions
  - Maud-owned board shell and fragments
  - initial Datastar-driven sheet, query, and focus shell state
  - create and edit sheets
  - filter builder GUI
  - focus mode detail loading
  - markdown preview in board card bodies
  - worklog start, stop, heartbeat, and offline stop queue
  - comments, assignees, hierarchy summaries, and resource references
  - sparse `widgetState` persistence
  - column drag resize through side borders
  - removal of the old width increment and decrement buttons
- runtime verification pending:
  - authenticated browser verification of focus mode on a fresh imported widget
  - authenticated browser verification of the filter GUI round trip on a fresh imported widget
  - authenticated browser verification of worklog and reconnect flows
  - final visual verification that the empty intermediate box and column-height box behavior are fully gone in the fresh imported widget
- final polish pending:
  - replace JS string-built filter, create, edit, and focus-action surfaces with Maud fragments and Datastar `data-bind`
  - replace manual DOM sync for status, lane layout, body modes, and active sheets with Datastar `data-text`, `data-class`, `data-style`, and `data-show`

- move `widgetState` persistence and bridge hydration further toward `data-on-signal-patch` and signal-first ownership
- simplify any remaining legacy widget runtime assumptions after the fresh imported widget is confirmed

9.16.29. Current code path migration checklist

Current widget path:

- `crates/web/src/sand/kanban_record_view/body.rs`
- `crates/web/src/sand/kanban_record_view/script.rs`
- `crates/web/src/application/kanban_render.rs`

Current `script.rs` responsibilities that should move to Datastar-first frontend state:

- `updateStatus()`, `setShellState()`, `clearShellState()`, and `setHeaderMetaFromContract()`: convert text, classes, disabled attributes, and copy into signals plus `data-text`, `data-class`, and `data-attr`
- `applyLaneLayout()`: keep resize math imperative, but move width and collapsed presentation into signal-driven `data-style` and `data-class`
- `applyCardModes()`: stop swapping body HTML with `innerHTML`; pre-render head, compact, and full body regions and switch them with Datastar visibility state
- `renderActiveFilters()`: replace string concatenation with fragment-owned chips or signal-patched chip markup
- `renderFilterSheet()`, `renderRecordForm()`, `renderCreateSheet()`, `renderEditSheet()`, and `renderFocusActionPanel()`: stop building HTML in JS; use Maud fragments with Datastar `data-bind`
- `syncActiveSheet()`, `openFilterSheet()`, `openCreateSheet()`, `openEditSheet()`, and `closeFocus()`: reduce these to signal transitions plus thin action calls
- `persistUi()` and host-state synchronization: keep the sparse shape, but move the trigger path toward `data-on-signal-patch` and signal-first bridge hydration
- `toggleWidgetStream()` and `togglePausedUpdates()`: keep the bridge calls, but make the button state and copy fully signal-driven
- `parseEventBlock()` and `consumeSseResponse()`: remove them when the official stream can emit Datastar fragments and signals directly

Current behavior already worth preserving:

- quantity-to-lane mapping
- optimistic move with rollback on failure
- host `widgetState` persistence for widths, collapse state, and body modes
- per-column scroll and width controls
- pause/resume controls
- compact mismatch and transport error states

Current responsibilities that should stay imperative JavaScript:

- pointer-driven lane resize

- HTML drag and drop and optimistic rollback
- scroll restoration and focus positioning after patches
- offline worklog stop queue, flush, and heartbeat loop
- minimal markdown or transport interop until the backend emits fully prepared Datastar patches

Preferred cleanup order against the current file:

1. replace JS HTML builders for filter, create, edit, and focus-action surfaces with Maud fragments plus Datastar bindings
2. replace manual status and shell DOM mutation with Datastar signals and computed state
3. move lane collapse, width styling, and body-mode presentation to `data-class` and `data-style`
4. move `widgetState` persistence to `data-on-signal-patch` with sparse host patches
5. keep drag/drop and resize isolated as the remaining imperative island
6. remove low-level SSE parsing once the stream emits Datastar fragments and signals directly
7. keep the offline worklog queue in JS unless a later host/runtime layer makes that imperative loop unnecessary

9.16.30. Final position

The official Record Kanban should be built as a server-shaped board with a rich local UI layer.

That means:

- server-shaped structure and data authority
- local reactive state for board ergonomics
- sidecar tables for richer task metadata
- normalized relations for users and hierarchy
- dedicated interval-based worklog storage
- host-owned auth and runtime state
- semantic actions where multi-table workflows become complex

9.17. Extensions

This document is a planning note for a future public extension hub for Lince.

Read `web_components.typ` first for:

- runtime contracts
- host and bridge assumptions
- runtime shape guidance
- standalone versus official widget packaging

Read `kanban.typ` for one concrete example of an official widget that is not only “some HTML”, but a hybrid runtime surface with a stronger contract.

Current code is more authoritative than this document when there is conflict. This note is intentionally incomplete and keeps unresolved issues visible instead of pretending they are solved.

9.17.1. Goal

The intended direction is:

- `lince` is a producer only
- `dna` is the public extension hub and canonical catalog
- `dna` owns checksum generation
- `dna` owns canonical writes, promotion, renames, and deletion
- community submission is open
- official publication is curated
- `upsert` is the normal write operation
- hard delete is allowed and explicit

This means the extension hub is not the same thing as widget authoring. It is the catalog, packaging, ingestion, and governance layer around authored artifacts.

9.17.2. Why this needs its own plan

The present web component documentation already distinguishes:

- official `sand/` widgets
- standalone imported widgets
- different runtime shapes

But an extension hub has to classify more than runtime style. It also has to classify:

- trust level
- package family
- ingestion path
- canonical storage
- compatibility and runtime expectations

That is why runtime descriptions alone are not enough.

9.17.3. Primary axes

The extension hub should model at least three different axes:

Axis	Examples	Why it matters
Channel	`official`, `community`	This is the hard trust and governance boundary. It should not be inferred from runtime style.

Family	`sand`, `db`	Different families have different payloads, validation rules, and user expectations.
Runtime requirements	`standalone iframe HTML`, `host-bound stream widget`, `foreign integration`, `engine-driven surface`	Useful for install warnings and human understanding, but too descriptive to be the main storage category.

Rules:

- channel is not runtime shape
- family is not runtime shape
- runtime shape is not a permission boundary
- runtime notes do not define the canonical folder layout

9.17.4. Relationship to runtime guidance

The runtime guidance in `web_components.typ` remains useful, but only as documentation for how a widget behaves.

They help answer:

- where truth lives
- how truth reaches the surface
- who shapes the UI
- where interaction state lives
- how foreign or portable the runtime is

They do not answer:

- whether an artifact is official or community
- whether the payload is a widget or a database example
- how checksums are owned
- how deletion or promotion works
- how the hub should store the artifact on disk

This matters because a runtime label can look like a package type when it is not.

Examples:

- `kanban` is a server-shaped board with rich local state, but that does not make “server-shaped” a package family
- a foreign integration widget and a client-owned renderer may both still be ordinary `sand` packages
- a database example does not need runtime-shape labeling at all

9.17.5. Kanban as warning

`kanban.typ` is useful here because it shows a real official widget is:

- hybrid
- host-configured
- instance-aware
- stream-backed
- action-oriented
- not portable in the same way as a standalone imported HTML widget

That means the extension hub cannot treat every extension as if it were just a static asset plus a friendly name.

At minimum, official widget metadata will likely need to describe:

- permissions
- required host configuration
- runtime contract style
- compatibility expectations
- whether the package is meant for import, direct hosting, or internal official use

9.17.6. Planned hub responsibilities

The intended split is:

- `lince` may produce code, HTML, source snapshots, or other raw extension payloads
- `lince` does not own canonical publication
- `dna` ingests submissions from a lounge area
- `dna` validates the submission according to family rules
- `dna` computes checksums
- `dna` performs upsert into the canonical tree
- `dna` performs hard delete when explicitly requested
- `dna` rebuilds the search catalog TOML

Community submission being “free to push” should mean:

- contributors can submit without maintainer curation before ingest into the community catalog
- `dna` still applies schema and safety validation
- the hub still has a canonical search TOML and checksum records
- maintainers may still rename, replace, or delete packages when they decide to

It should not mean:

- no validation
- no naming rules
- no canonical metadata

9.17.7. Near-term on-disk direction

The practical near-term direction should be family-first and channel-explicit on disk. Packages should be sharded by the first two letters of the package name, similar to the broad shape used by `nixpkgs`. There should be no JSON catalog file. Instead, the hub should maintain one search TOML for `sand`.

Suggested direction:

```
dna/
  lounge/
    <package-name>/
      index.html?        # required for sand packages
      sand.toml?         # sand package metadata
      *.sqlite
      ...
  sand/
    catalog.toml
    official/
      <prefix>/
        <package-name>/
          <package-name>.html
          <package-name>_metadata.html
          sand.toml
          sha256.txt
    community/
      <prefix>/
        <package-name>/
          <package-name>.html
          <package-name>_metadata.html
          sand.toml
          sha256.txt
  db/
    official/
      <prefix>/
        <package-name>/
          ...
          sha256.txt
    community/
      <prefix>/
        <package-name>/
          ...
          sha256.txt
```

Example:

```
sand/official/he/hello_world/
```

Here the package name is the stable package identity used in the path.

Rules:

- package names are first-come, first-served

- the package name is the stable package id
- package names must be `lower_snake_case`
- there must not be two packages with the same name in the same family across `official` and `community`
- `official` has naming priority over `community`
- when a community package conflicts with an official package, the community package must be renamed
- maintainers may rename community or official packages when needed
- the prefix is derived from the first two letters of the `lower_snake_case` package name
- if the package name has fewer than two letters, use the available letters
- runtime notes do not affect the on-disk layout

This keeps the hard official versus community split visible while avoiding one flat directory with too many entries.

9.17.8. Catalog TOML

The hub should keep one search TOML for `sand`.

Suggested direction:

`sand/catalog.toml`

Lookup rule:

1. Normalize the package name.
2. Open `sand/catalog.toml`.
3. Read the package record from there instead of scanning the whole tree.

Suggested `sand` catalog shape:

```toml
[packages.hello_world]
title = "Hello World"
description = "Example package"
path = "community/he/hello_world"
```

Rules:

- the catalog is the fast lookup layer
- the canonical package directory remains the artifact source of truth
- `sand` should have only one search TOML in the first version
- the catalog must be updated on create, upsert, rename, promotion, and delete

9.17.9. Rename history

Package renames should be recorded.

The simplest direction is a per-family migrations ledger such as:

```
sand/migrations.txt
db/migrations.txt
```

Each rename entry can be append-only and human-readable, for example:

```
2026-03-29T12:00:00Z community/tes -> community/test
2026-03-29T12:10:00Z community/hello_world -> official/hello_world
```

This does not need to be a heavy migration system. It only needs to preserve package rename history in a form that is easy to inspect.

9.17.10. Package metadata

The hub should not use `manifest.json` as the package metadata file.

Use family-specific TOML files instead:

- `sand.toml` for web components

For `sand`, the package should normally contain:

- `index.html`
- `sand.toml`
- optional extra assets if the package format later allows them

For now, extra assets should be allowed broadly. That includes arbitrary supporting files and directories. Asset hygiene can stay as a manual maintainer responsibility rather than an automated ingest restriction.

In canonical storage, the ingested package should normally contain:

- `<package-name>.html`
- `<package-name>_metadata.html`
- `sand.toml`
- optional extra assets if the package format later allows them

The family-specific TOML file is where package metadata belongs. There is no need for one generic package manifest for all families in the first version.

For `db`, metadata remains intentionally unspecified for now. At this stage, `db` is just storage for SQLite database files and is not the center of the automated metadata plan.

9.17.10.1. `sand.toml`

`sand.toml` should be closer in spirit to `Cargo.toml` package metadata than to an ad-hoc JSON manifest.

Suggested first version fields:

- `name`: required
- `channel`: required, either `official` or `community`
- `version`: required
- `author`: required
- `description`: required
- `title`: optional

- `icon`: optional
- `details`: optional
- `initial_width`: optional
- `initial_height`: optional
- `permissions`: optional list of strings
- `tags`: optional list of strings

Rules:

- `name` must be `lower_snake_case`
- `name` must be at most `200` characters
- `name` must match `^[a-z][a-z0-9]*(?:_[a-z0-9]+)*$`
- `name` must not be one of the reserved names `official`, `community`, `sand`, `db`, `lounge`, `map`, `migrations`, `con`, `prn`, `aux`, `nul`, `com1` through `com9`, or `lpt1` through `lpt9`
- `version` must be valid semantic versioning text
- `initial_width`, if present, must be an integer from `1` to `6`
- `initial_height`, if present, must be an integer from `1` to `6`
- the lounge directory name and `sand.toml` `name` should agree

`sand.toml` should be the source of truth for package name and channel during ingest.

Defaults for metadata-expanded HTML generation:

- `title` defaults to a humanized form of `name`
- `details` defaults to `description`
- `initial_width` defaults to `4`
- `initial_height` defaults to `3`
- `permissions` defaults to `[]`

9.17.10.2. Future compatibility metadata

Simple standalone `sand` packages do not need much compatibility metadata beyond ordinary package fields.

Official or host-bound widgets may later need optional compatibility fields such as:

- `required_permissions`: optional list of required host permissions
- `required_host_meta`: optional list of required host metadata keys such as `server_id` or `view_id`
- `host_contract_version`: optional string describing the expected host runtime contract
- `min_lince_version`: optional semantic version describing the minimum supported Lince version

If these fields are absent, the conservative assumption should be that the package is a simpler standalone-style package rather than a strongly host-bound official runtime surface.

If these fields are present:

- ingest should validate their syntax and basic semantics

- hub ingest should reject malformed compatibility metadata
- runtime compatibility should be enforced when the package is installed or executed, not by guessing during catalog ingest

9.17.10.3. Sand HTML files

When a `sand` package is ingested:

- the lounge `index.html` should be copied into canonical storage as `<package-name>.html`
- `dna` should also generate `<package-name>_metadata.html`

The purpose of `<package-name>_metadata.html` is to provide the current embedded metadata block expected by the existing standalone widget contract.

When generating that embedded metadata block:

- manifest `icon` should come from `sand.toml` `icon`
- manifest `title` should come from `sand.toml` `title`, or fall back to a humanized form of `name`
- manifest `author` should come from `sand.toml` `author`
- manifest `version` should come from `sand.toml` `version`
- manifest `description` should come from `sand.toml` `description`
- manifest `details` should come from `sand.toml` `details`, or fall back to `description`
- manifest `initial_width` should come from `sand.toml` `initial_width`, or default to `4`
- manifest `initial_height` should come from `sand.toml` `initial_height`, or default to `3`
- manifest `permissions` should come from `sand.toml` `permissions`, or default to `[]`

If the source HTML already contains embedded metadata, `dna` may still regenerate the metadata version so that the canonical metadata-expanded file is deterministic.

This keeps:

- the original author HTML, preserved under the package name
- a metadata-expanded HTML file, compatible with the current embedded-manifest expectations

9.17.11. Package identity

The simplest direction is:

- the package id is the package name
- the package id does not change
- upsert replaces the current canonical package at that path
- hard delete removes that path

This is a deliberately simple model. It avoids introducing a second hidden identifier before there is a real reason for one.

9.17.12. Validation

For now, only `sand` needs a stronger validation story.

The minimum first-version validation for `sand` should be:

- `index.html` exists
- `sand.toml` exists
- `index.html` passes a loose HTML validity check through `HTMLHint`
- `sand.toml` matches the expected schema
- the package path and the `name` declared in `sand.toml` agree
- the `channel` declared in `sand.toml` is valid
- the `version` declared in `sand.toml` is valid semantic version text
- `permissions`, if present, is a list of strings
- the package `name` fits `lower_snake_case`
- the package `name` is at most `200` characters

The HTML validation should stay loose. It only needs to confirm that the document is valid enough to be treated as HTML, more like an HTML linter than a strict runtime certification.

The ingest command should show short user-facing error messages on validation failure. Recommended messages:

- `missing sand.toml: sand packages require sand.toml`
- `missing index.html: sand packages require index.html`
- `invalid package name 'Hello World': use lower_snake_case with at most 200 characters`
- `invalid channel 'beta': expected 'official' or 'community'`
- `invalid version '1.0': expected semantic versioning such as '1.0.0'`
- `version must increase: current canonical version is '0.2.0'`
- `community package 'hello_world' conflicts with official package 'hello_world': choose another name`
- `html validation failed: document is not valid enough for sand ingest`
- `reserved package name 'official': choose another name`

`HTMLHint` should be the first-version linter. The practical tooling direction is to have `mise ingest` install or expose `HTMLHint` so the ingest command has the linter available.

9.17.13. Checksums

`dna` owns checksum generation.

The first version does not need checksum history or lineage tracking. It only needs the current checksum of the canonical package written to `sha256.txt`.

The checksum should be computed across the canonical package files in alphabetical relative-path order.

The deterministic hashing rule should be:

- sort canonical file paths alphabetically

- feed each relative path and its file bytes into the hash in that order

This avoids ambiguity once a package contains more than one file.

On upsert:

- require a semantic version bump relative to the current canonical package
- recompute the checksum
- overwrite `sha256.txt`
- update the corresponding catalog entry

On hard delete:

- remove the package directory
- remove the corresponding catalog entry
- leave the filesystem and migrations ledger as the remaining source of truth

9.17.14. Lounge ingestion

The ingest path should be `lounge/`, not `.lounge/`.

The simplest direction is:

- `lounge/<package-name>/...`

The ingest command can infer the family from the package contents.

The first version inference rule should be:

- if `sand.toml` is present, treat the package as `sand`
- `sand` requires `index.html`
- if `sand.toml` is absent, the package is not part of the automated sand ingest flow

For now, `db` remains out of the automated metadata validation path. Maintainers may still place SQLite database files in the `db/` tree manually.

That is enough for now because `sand` is the only fully-specified automated package flow in this plan.

9.17.15. Practical operations

9.17.15.1. Create

For creation:

1. Put the package under `dna/lounge/<package-name>/`.
2. Run the `dna` ingest command.
3. `dna` treats the package as `sand` if `sand.toml` is present.
4. `dna` validates the package according to the `sand` rules.
5. `dna` reads the target channel from package metadata.
6. `dna` checks for a name collision in that family across `official` and `community`.

7. If the package is community and the name is taken by an official package, ingest must reject it until the community package uses another name.
8. `dna` computes the checksum.
9. `dna` creates the canonical directory if it does not already exist.
10. `dna` writes the canonical package into the chosen family and channel.
11. `dna` writes or updates the corresponding catalog entry.

This gives the operator the simple workflow of "put it somewhere, run one command, done".

9.17.15.2. Upsert

For updates:

1. Reuse the same package name.
2. Submit the new package contents.
3. Run ingest again.
4. `dna` validates again.
5. `dna` reads the target channel from package metadata.
6. `dna` requires the submitted semantic version to be greater than the current canonical version.
7. `dna` replaces the canonical package at the same path.
8. `dna` recomputes checksum.
9. `dna` updates the corresponding catalog entry.

9.17.15.3. Rename

For renames:

1. A maintainer chooses the new package name.
2. The rename must include a semantic version bump because the package metadata and canonical filenames change.
3. `dna` moves the canonical package to the new path.
4. `dna` updates the package metadata name.
5. `dna` appends a line to the relevant family migration file.
6. `dna` updates the old and new catalog entries.

This is the main administrative escape hatch for collision handling and curator intervention.

9.17.15.4. Promotion

For promotion from `community` to `official`:

1. A maintainer selects the existing community package.
2. If the target official name is already used by another official package, promotion is rejected.

3. If an official ingest or promotion takes a name currently held by `community`, `dna` should auto-rename the displaced community package, patch-bump it, and write that rename to the migration file.
4. Promotion must include a semantic version bump because the package metadata changes.
5. `dna` moves the package from `community` to `official`.
6. `dna` updates the package metadata channel.
7. `dna` appends a line to the migration file recording the channel move.
8. `dna` updates the catalog entry.

Promotion should be a move, not a copy. The package keeps the same name unless a maintainer explicitly renames it.

9.17.15.5. Hard delete

For deletion:

1. Submit a delete request or invoke an explicit delete command with the family, channel, and package name.
2. `dna` removes the canonical package directory.
3. `dna` removes the catalog entry.

The plan does not try to soften the consequences of hard delete.

9.17.16. Recommended practical direction for now

The most practical direction today is:

- keep only `sand` and `db`
- store packages by family, then channel, then two-letter prefix, then package name
- use `sand.toml`
- use one `sand/catalog.toml` file for search
- treat the package name as the stable id
- keep package names unique per family, with `official` taking precedence over `community`
- use `lounge/` as the operator drop zone
- keep `official` and `community` as the hard boundary
- let `dna` own checksum and canonical writes
- let `lince` remain a producer only
- require `lower_snake_case` names
- require semantic versions for `sand`
- use loose `HTMLHint` validation for `sand`
- keep runtime expectations in documentation and compatibility fields
- treat `db` as manual storage for now rather than part of the automated metadata flow

This is much simpler than a generalized hub model, and it matches the current stage of the project better.

9.17.17. Criticisms of the current direction

Several criticisms remain even after simplifying the design:

- First-come, first-served naming is simple, but the real answer to disputes is maintainer override rather than a neutral policy.
- Using the package name as the stable id is pragmatic, but it means renames need explicit migration bookkeeping.
- The two-letter prefix sharding is practical, but it makes names and path normalization a hard policy boundary instead of a forgiving one.
- Official widgets like Kanban still need richer runtime metadata than a generic standalone HTML package, even if the storage layout stays simple.
- A single `sha256.txt` is enough operationally, but it intentionally gives up richer provenance.
- A single `sand/catalog.toml` is simpler than many map files, but it becomes one shared file that changes on every catalog mutation.
- Allowing arbitrary assets keeps the system flexible, but it also pushes all asset hygiene and review onto maintainers.

9.17.18. Concrete decisions

The remaining policy points should be treated as decided for the current plan:

- invalid package names are rejected if they fail the `lower_snake_case` pattern, exceed `200` characters, or use a reserved name
- official-widget compatibility metadata is optional, but if present it must pass schema validation during ingest
- `sand.toml` may later grow `homepage`, `repository`, and compatibility fields without changing the current package identity model
- every canonical package change requires a semantic version bump, including rename and promotion
- if `db` later becomes automated, it should get its own metadata file and, if needed, its own search TOML

9.17.19. Working conclusion

The extension hub can be much simpler than the first draft.

The current best direction is:

- `sand` and `db` only
- `sand.toml` drives the automated ingest flow
- one `sand/catalog.toml` avoids full directory scans
- package name as stable id
- two-letter path sharding
- one `lounge/` drop zone
- `dna` owns checksum and canonical writes

- `official` and `community` stay explicit
- `sand` gets its channel from `sand.toml`
- promotion is a move, renames are recorded in migrations, and both update `sand/catalog.toml`

Runtime guidance still matters for understanding widget behavior, but it does not need a dedicated taxonomy field or storage role in this plan.

9.17.20. Next board step

After the ingest and catalog rules are in place, the next practical UI step should be board-side package pickup from `github.com/lince-social/dna` on `main`.

The intended direction is:

1. Add one more choice to the existing `Add card` popover for remote catalog pickup.
2. Label it something direct such as `Hub` or `DNA`.
3. Keep the existing `Importar` and `Local` flows unchanged.
4. Open a new Maud-rendered modal instead of navigating away or inventing a second screen.

The modal should follow the same broad shape as the current local catalog modal:

- search field at the top
- short catalog summary text
- scrollable list of matching package names
- one clear action per result to install and add the card

Suggested HTML shape in the page template:

```
button#add-card-dna-button
div#dna-packages-modal-backdrop
section#import-modal.import-modal--catalog
input#dna-packages-search
div#dna-packages-summary
div#dna-package-list
```

This should be rendered in Maud next to the existing local catalog modal, not assembled ad-hoc in client JavaScript.

The search path should use one catalog TOML rather than full repository scans.

Practical rule:

1. Normalize the user query to the package-name shape.
2. Take the first two letters.
3. Fetch `sand/catalog.toml` from `main`.
4. Filter the package records in that TOML file on the client or through the backend proxy.
5. Render the matching package rows in the modal.

The first version can stay simple:

- search by package name first

- treat empty search as “show nothing yet” or “type at least two letters”
- only target `sand`
- prefer `official` entries first when both channels are ever shown together

The fetch source should be the `main` branch of the public `dna` repository. The safer runtime path is a backend proxy in `lince`, even if the upstream source is GitHub, because that avoids client-side CORS assumptions and keeps remote fetching inside the server boundary.

Suggested remote sources:

- `sand/catalog.toml` for search
- `sand/<channel>/<prefix>/<package-name>/sand.toml` for package metadata
- `sand/<channel>/<prefix>/<package-name>/<package-name>_metadata.html` for installation

The installation path should reuse the current imported-widget flow as much as possible.

Practical direction:

1. The user selects a package row in the modal.
2. `lince` fetches that package’s `sand.toml` and `<package-name>_metadata.html` from `dna/main`.
3. `lince` turns that into the same preview payload shape already used for local/imported widgets.
4. The user sees the normal preview and confirms.
5. On confirm, `lince` stores the downloaded widget into the local package catalog and creates the card.

This means the remote hub pickup should land in the same local installed-catalog path as other widgets after download. The hub is the source, but the board still works with a local installed package once the user adds it.

The shortest implementation sequence should be:

1. add `Hub` or `DNA` to the `Add card` popover
2. add the Maud modal shell
3. add backend endpoints that proxy GitHub `main` TOML and HTML fetches
4. add client search + result rendering using `sand/catalog.toml`
5. add preview + confirm using the existing imported-widget card creation path

9.17.20.1. Board UX plan

The board-side experience should be:

1. Enter edit mode.
2. Open `Add card`.
3. Choose `DNA`.
4. The modal opens with an empty result list and a message such as `Digite pelo menos duas letras para buscar no hub.`
5. After the user types two or more characters, the board searches the hub.

6. The modal shows matching package rows with enough metadata to decide quickly.
7. Selecting a row opens the same preview surface already used for imported widgets.
8. Confirming the preview downloads the package into the local installed catalog and creates the card.

The first version should keep the interaction narrow:

- one search field
- one result list
- one preview-confirm path
- no multi-select
- no background synchronization

9.17.20.2. DNA search files

For the board flow, `dna` should expose one TOML search file in `main`:

`sand/catalog.toml`

The purpose of `sand/catalog.toml` is:

- provide the search rows for the modal
- avoid repository directory scans
- keep one compact lookup file for the whole `sand` family

Suggested shape:

```toml
[packages.hello_world]
title = "Hello World"
description = "Example package"
path = "official/he/hello_world"
```

The package name lives in the TOML key. The row metadata for search is only:

- `title`
- `description`
- `path`

9.17.20.3. Search behavior

The search contract should be simple and deterministic:

1. trim the query
2. lowercase it
3. normalize spaces and dashes into _
4. reject characters that do not fit package-name search
5. if fewer than two characters remain, do not hit the network
6. fetch `sand/catalog.toml`
7. filter the package rows by package name first
8. if desired, also match `title` and `description`

Result ordering should be:

1. exact package-name prefix match
2. package-name substring match
3. title match
4. description match
5. alphabetical by title

The first version should fetch the whole `sand/catalog.toml` file for each search refresh or from cache. That keeps the search contract simple.

9.17.20.4. Modal contents

Each result row in the Maud modal should show:

- icon
- title
- package name
- short description

Each row should have one explicit action:

- `Preview`

The modal should not install immediately from the search list. Preview first, then confirm through the existing preview modal.

9.17.20.5. Backend route plan

The backend should own all remote GitHub fetches. The browser should never fetch raw GitHub URLs directly.

Suggested new API surface:

```
GET  /api/packages/dna/catalog
GET  /api/packages/dna/search?q=<query>
GET  /api/packages/dna/{channel}/{package_name}/preview
POST /api/packages/dna/{channel}/{package_name}/install
```

Responsibilities:

- `catalog`: fetch and cache `sand/catalog.toml`
- `search`: normalize the query, fetch `sand/catalog.toml`, filter rows, and return lightweight summaries
- `preview`: fetch `sand.toml` plus `<package-name>_metadata.html`, parse them into the same preview payload shape already used by `/api/packages/local/{package_id}` and `/api/packages/preview`
- `install`: fetch the same remote package, persist it into the local widget catalog, and return the installed preview payload

This keeps the remote-hub logic inside the backend and lets the front-end reuse the existing package UI model.

9.17.20.6. Remote source rules

The first version should fetch only from:

- repository: `github.com/lince-social/dna`
- branch: `main`
- family: `sand`

The backend should translate those logical paths to raw file fetches from GitHub.

The expected remote files for a package are:

- `sand.toml`
- `<package-name>_metadata.html`

The board should not download the plain `<package-name>.html` file for installation in the first version. The metadata-expanded HTML is the correct transport because it already satisfies the current embedded-manifest package contract used by local install and preview flows.

9.17.20.7. Local install rule

When `lince` installs a package downloaded from `dna`:

1. fetch `<package-name>_metadata.html`
2. parse it as a normal Lince package
3. persist it locally under the filename `<package-name>.html`

This rule is important. The remote transport file is `<package-name>_metadata.html`, but the local installed filename should still be `<package-name>.html`. Otherwise the local package id would drift to `<package-name>_metadata`, which is wrong.

The installed package should land in the same local catalog used today for uploaded widgets:

`~/.config/lince/web/widgets/<package-name>.html`

After that, the package behaves like any other locally installed widget.

9.17.20.8. Collision and overwrite rule

For the first version, the simplest local rule should be:

- if the package is already installed locally and the downloaded version is newer, overwrite the local installed copy
- if the package is already installed locally and the downloaded version is the same, replace it anyway if the user explicitly installs it from the hub

- if the package exists only as a built-in official sand widget, allow the downloaded local copy to shadow it

This is explicit user intent. Choosing a package from the hub is enough reason to let the local installed copy win over the built-in rendered official copy.

9.17.20.9. Caching rule

The backend should cache remote TOML and HTML responses in memory.

The first version can stay simple:

- cache `catalog.toml` for 5 minutes
- cache preview payloads for 1 minute

Cache invalidation does not need to be perfect. The branch is `main`, not a versioned immutable archive, so short-lived stale reads are acceptable for the first version.

9.17.20.10. Error handling

The modal and preview flow should surface short concrete errors:

- `Falha ao buscar o catalogo do hub.`
- `Nenhum pacote corresponde a essa busca.`
- `Falha ao baixar o pacote selecionado.`
- `O pacote remoto nao possui metadados validos para instalacao.`
- `Falha ao instalar o pacote do hub no catalogo local.`

The modal should stay open on search and preview errors. Only a successful install should close the remote modal and continue into the normal add-card success path.

9.17.20.11. Data shape reuse

The implementation should reuse the existing local package data shapes as much as possible.

Practical rule:

- search rows should look like a minimal summary built from package name plus `title`, `description`, and `path`
- preview responses should look like the existing `PackagePreview`
- install responses should also return `PackagePreview`

This keeps the new `DNA` path close to the existing local/imported widget path and avoids inventing a second package model in the front-end.

9.17.20.12. Implementation order

The practical implementation order should be:

1. extend `dna` ingest/rebuild so it writes `sand/catalog.toml`
2. add `DNA` to the add-card popover in Maud
3. add the Maud modal shell for remote search
4. add backend routes for `catalog`, `search`, `preview`, and `install`
5. add backend GitHub fetch + in-memory cache
6. add client-side search state and result rendering from `sand/catalog.toml`
7. route row click into the existing preview-confirm flow
8. persist the downloaded package locally as `<package-name>.html`
9. create the card using the existing imported-widget card creation path

Live

10. The Death of Lince

Death is only the beginning

— The Mummy (1999)

Most software is repetitive Create, Read, Update, Delete (CRUD) operations. They exist as islands on the big internet sea. Lince tries to build a Pangea.

We can create a good general base for basic CRUD software, and continue to expand beyond that. The diversity in what can be created with it is the real power.

People will have access to a free and open source todo app, event planner, financial analysis platform, industrial production planning and more; all with a change of data (DNA) and maybe some extensions.

Once the operational part of life is modeled, automated and interconnected, the effects of having built a robust and generic base for interaction with systems will start to show. If there is no incentive to rebuild a management system/platform (because there is a clearly G.O.A.T.ed one), more efforts will have to be focused on new highly specific innovative and optmizing ideas. People will enjoy having previously centralized, private and paywalled capabilities now in close reach for free.

Think of Amazon and Uber, they are companies that get a cut, to orchestrate something that computationally a global peer-to-peer network personal devices could easily manage. Think of how much your much time your devices spend processing your online purchases and how much it could be plugged into the LNHM (Lince-Net-Hive-Mind).

Amazon is a platform for Records of item contributions, your purchases are Transfer Proposals. The rides you make on Uber are Transfer Proposals of money for transportation from point A to B. Any automation they make inside their platforms is limited to the data they have on you, which is a miniscule amount of your whole life (at least I hope it is). If you where to set up hacky ways to use those platforms automatically to fit your needs, you would be doing it to every app you have, connecting them in ways they where not meant to, possibly programming something to organize all those changing external apps. It doesn't need to be that inefficient, you can have one central program that you can trust to work offline for you, acting just the way you want, featureful, built by many people, for it is the main one. The model of Lince can be a superset of many apps, and can spark many different markets and support people with it's usage.

Lince is actually not the app you install, that's a Lince Cell. When you have a group of Cells (or p2p nodes) you have a Lince Organ; people interacting to meed their needs (or others'). Lince is actually when all Cells and Organs work together to satiate everyone's Needs.

The assumption is that an app that is useful for a lot of cases and incentivizes people to help eachother, with no extra friction, is the best use of technology we can have. It will not solve anything by itself, but is is the best tool in the software part of the world.

10.1. Serious Mode

Like every good contribution, the goal is to not have to contribute anymore; because it was a one time need, or the frequent need has been solved.

Lince is a contribution to the need of having Lince, of having all production possibly connected, so there is predictability in resource usage and logistics coordination.

It's a contribution to the disorganized construction of solutions glued by the internet, centralized in platforms that worsen by the day.

It's a contribution to understanding the whole of possibilities of how you can connect with other people, after seeing needs you didnt know others had, sparking ideas of contributions you didn't know you could make.

The death of Lince is the death of the need of Lince. If there is ever any configuration of resources that makes Lince useless, then it's job is fullfilled, there is nothing else to be done, no commit, no push, just vibes. Untill then, a system for self organization that can turn into a dance of the world is, to some, something useful and exciting to build.

A team is being assembled, to fight the battle of killing Lince. Will you join the cult?